



Red Hat OpenShift AI Self-Managed 3.5

Installing and uninstalling OpenShift AI Self-Managed

Install and uninstall OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.5 Installing and uninstalling OpenShift AI Self-Managed

Install and uninstall OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Install and uninstall OpenShift AI Self-Managed on your OpenShift cluster.

Table of Contents

PREFACE	4
CHAPTER 1. ARCHITECTURE OF OPENSIFT AI SELF-MANAGED	5
CHAPTER 2. UPDATE CHANNELS	7
CHAPTER 3. INSTALLATION AND DEPLOYMENT OF OPENSIFT AI	9
3.1. REQUIREMENTS FOR OPENSIFT AI SELF-MANAGED	9
3.1.1. Platform requirements	9
3.1.2. Component requirements	12
3.2. CONFIGURE CUSTOM NAMESPACES	13
3.3. INSTALL THE RED HAT OPENSIFT AI OPERATOR	15
3.3.1. Installing the Red Hat OpenShift AI Operator by using the CLI	15
3.3.2. Installing the Red Hat OpenShift AI Operator by using the web console	18
3.4. INSTALL AND MANAGE RED HAT OPENSIFT AI COMPONENTS	20
3.4.1. Installing Red Hat OpenShift AI components by using the CLI	20
3.4.1.1. Configure OAuth proxy sidecar resources for KServe	24
3.4.2. Installing Red Hat OpenShift AI components by using the web console	26
3.4.3. Update the installation status of Red Hat OpenShift AI components by using the web console	30
3.4.4. View installed OpenShift AI components	32
CHAPTER 4. CONFIGURE PIPELINES WITH YOUR OWN ARGO WORKFLOWS INSTANCE	33
CHAPTER 5. INSTALL THE DISTRIBUTED WORKLOADS COMPONENTS	35
CHAPTER 6. ACCESS THE DASHBOARD	38
CHAPTER 7. ENABLE ACCELERATORS	39
CHAPTER 8. WORK WITH CERTIFICATES	41
8.1. UNDERSTANDING HOW OPENSIFT AI HANDLES CERTIFICATES	41
8.2. ADDING CERTIFICATES	42
8.3. ADDING CERTIFICATES TO A CLUSTER-WIDE CA BUNDLE	43
8.4. ADDING CERTIFICATES TO A CUSTOM CA BUNDLE	44
8.5. USING SELF-SIGNED CERTIFICATES WITH OPENSIFT AI COMPONENTS	45
8.5.1. Access S3-compatible object storage with self-signed certificates	45
8.5.2. Configure a certificate for pipelines	47
8.5.3. Configure a certificate for workbenches	50
8.5.4. Using the cluster-wide CA bundle for the model serving platform	52
8.5.5. CA bundle configuration for OGX	53
8.5.6. Configure a CA bundle for OGX	54
8.5.7. CA bundle configuration for OGX	57
8.5.7.1. caBundle subfields	57
8.5.7.2. Configuration examples	58
8.5.7.3. Validation rules	59
8.5.7.4. Limits	59
8.5.7.5. CA bundle validation failure conditions	59
8.5.7.6. Using the CA bundle from client code	60
8.6. MANAGE CERTIFICATES WITHOUT THE RED HAT OPENSIFT AI OPERATOR	61
8.7. REMOVING THE CA BUNDLE	62
8.7.1. Remove the CA bundle from all namespaces	62
8.7.2. Remove the CA bundle from a single namespace	63
CHAPTER 9. VIEW LOGS AND AUDIT RECORDS	65

9.1. CONFIGURE THE OPENSIFT AI OPERATOR LOGGER	65
9.2. VIEW AUDIT RECORDS	67
CHAPTER 10. TROUBLESHOOTING REFERENCE: INSTALLATION	69
10.1. INSTALLING CERTAIN OPERATORS REDIRECTS TO CLUSTEREXTENSIONS PAGE	69
10.2. THE RED HAT OPENSIFT AI OPERATOR CANNOT BE RETRIEVED FROM THE IMAGE REGISTRY	70
10.3. OPENSIFT AI DOES NOT INSTALL ON UNSUPPORTED INFRASTRUCTURE	70
10.4. THE CREATION OF THE OPENSIFT AI CUSTOM RESOURCE (CR) FAILS	71
10.5. THE CREATION OF THE OPENSIFT AI NOTEBOOKS CUSTOM RESOURCE (CR) FAILS	72
10.6. THE OPENSIFT AI DASHBOARD IS NOT ACCESSIBLE	72
10.7. REINSTALLING OPENSIFT AI FAILS WITH AN ERROR	73
10.8. THE DEDICATED-ADMINS ROLE-BASED ACCESS CONTROL (RBAC) POLICY CANNOT BE CREATED	74
10.9. THE ODH PARAMETER SECRET DOES NOT GET CREATED	74
CHAPTER 11. UNINSTALL RED HAT OPENSIFT AI SELF-MANAGED	76
11.1. UNDERSTANDING THE UNINSTALLATION PROCESS	76
11.2. UNINSTALLING OPENSIFT AI SELF-MANAGED BY USING THE CLI	77

PREFACE

Learn how to use both the OpenShift CLI (**oc**) and web console to install Red Hat OpenShift AI Self-Managed on your OpenShift cluster. To uninstall the product, learn how to use the recommended command-line interface (CLI) method.



NOTE

Red Hat does not support installing more than one instance of OpenShift AI on your cluster.

Red Hat does not support installing the Red Hat OpenShift AI Operator on the same cluster as the Red Hat OpenShift AI Add-on.

CHAPTER 1. ARCHITECTURE OF OPENSIFT AI SELF-MANAGED

Red Hat OpenShift AI Self-Managed is an Operator that is available in a self-managed environment, such as Red Hat OpenShift Container Platform, or in Red Hat-managed cloud environments such as Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP), Red Hat OpenShift Service on Amazon Web Services (ROSA classic or ROSA HCP), or Microsoft Azure Red Hat OpenShift.

OpenShift AI integrates the following components and services:

- At the service layer:

OpenShift AI dashboard

A customer-facing dashboard that shows available and installed applications for the OpenShift AI environment as well as learning resources such as tutorials, quick starts, and documentation. Administrative users can access functionality to manage users, clusters, workbench images, and model-serving runtimes. Data scientists can use the dashboard to create projects to organize their data science work.

Model serving

Data scientists can deploy trained machine-learning models to serve intelligent applications in production. After deployment, applications can send requests to the model using its deployed API endpoint.

AI pipelines

Data scientists can build portable machine learning (ML) workflows with AI pipelines by using Docker containers. With AI pipelines, data scientists can automate workflows as they develop their data science models.

Jupyter (self-managed)

A self-managed application that allows data scientists to configure a basic standalone workbench and develop machine learning models in JupyterLab.

Distributed workloads

Data scientists can use multiple nodes in parallel to train machine-learning models or process data more quickly. This approach significantly reduces the task completion time, and enables the use of larger datasets and more complex models.

Retrieval-Augmented Generation (RAG)

Data scientists and AI engineers can leverage Retrieval-Augmented Generation (RAG) capabilities provided by the integrated OGX Operator. By combining large language model inference, semantic retrieval, and vector database storage, data scientists and AI engineers can obtain tailored, accurate, and verifiable answers to complex queries based on their own datasets within a project.

- At the management layer:

The Red Hat OpenShift AI Operator

A meta-operator that deploys and maintains all components and sub-operators that are part of OpenShift AI.

When you install the Red Hat OpenShift AI Operator in the OpenShift cluster using the predefined projects, the following new projects are created:

- The **redhat-ods-operator** project contains the Red Hat OpenShift AI Operator.

- The **redhat-ods-applications** project includes the dashboard and other required components of OpenShift AI.
- The **rhods-notebooks** project is where basic workbenches are deployed by default.

You can specify custom projects if needed. You or your data scientists must also create additional projects for the applications that will use your machine learning models.

Do not install independent software vendor (ISV) applications in namespaces associated with OpenShift AI.

CHAPTER 2. UPDATE CHANNELS

You can use update channels to specify which Red Hat OpenShift AI minor version you intend to update your Operator to. Update channels also allow you to choose the timing and level of support your updates have through the **fast**, **stable**, **stable-x.y**, **eus-x.y**, and **alpha** channel options.

The subscription of an installed Operator specifies the update channel, which is used to track and receive updates for the Operator. You can change the update channel to start tracking and receiving updates from a newer channel. For more information about the release frequency and the lifecycle associated with each of the available update channels, see the *Red Hat OpenShift AI Self-Managed Life Cycle* Knowledgebase article.

Channel	Support	Release frequency	Recommended environment
fast or fast-x.y	One month of full support	Every month	Production environments with access to the latest product features. Select this streaming channel with automatic updates to avoid manually upgrading every month. NOTE: OpenShift AI 3.0 is available only through the fast-3.x channel, not the general fast channel.
stable	Three months of full support	Every three months	Production environments with stability prioritized over new feature availability. Select this streaming channel with automatic updates to access the latest stable release and avoid manually upgrading.
stable-x.y	Seven months of full support	Every three months	Production environments with stability prioritized over new feature availability. Select numbered stable channels (such as stable-2.25) to plan and upgrade to the next stable release while keeping your deployment under full support.
eus-x.y	Seven months of full support followed by Extended Update Support for eleven months	Every nine months	Enterprise-grade environments that cannot upgrade within a seven month window. Select this streaming channel if you prioritize stability over new feature availability.

Channel	Support	Release frequency	Recommended environment
alpha	One month of full support	Every month	Development environments with early-access features that might not be functionally complete. Select this channel to use early-access features to test functionality and provide feedback during the development process. Early-access features are not supported with Red Hat production service level agreements (SLAs). For more information about the support scope of Red Hat Technology Preview features, see <i>Technology Preview Features Support Scope</i>. For more information about the support scope of Red Hat Developer Preview features, see <i>Developer Preview Features Support Scope</i>.

**NOTE**

The **embedded** and **beta** channels are legacy channels that will be removed in a future release. Do not select the **embedded** or **beta** channels for a new Operator installation.

Additional resources

- [Red Hat OpenShift AI Self-Managed Life Cycle](#)
- [Technology Preview Features Support Scope](#)
- [Developer Preview Features Support Scope](#)

CHAPTER 3. INSTALLATION AND DEPLOYMENT OF OPENSIFT AI

Red Hat OpenShift AI is a platform for data scientists and developers of artificial intelligence (AI) applications. It provides a fully supported environment that lets you rapidly develop, train, test, and deploy machine learning models on-premises and/or in the public cloud.

OpenShift AI is provided as a managed cloud service add-on for Red Hat OpenShift or as self-managed software that you can install on-premise or in the public cloud on OpenShift.

For information about installing OpenShift AI as self-managed software on your OpenShift cluster in a disconnected environment, see *Installing and uninstalling OpenShift AI Self-Managed in a disconnected environment*.

Installing OpenShift AI involves the following high-level tasks:

1. Confirm that your OpenShift cluster meets all requirements.
2. Install the Red Hat OpenShift AI Operator.
3. Install OpenShift AI components.
4. Complete any additional configuration required for the components you enabled.
5. Configure user and administrator groups to provide user access to OpenShift AI.
6. Access the OpenShift AI dashboard.

Additional resources

- [Installing and uninstalling OpenShift AI Self-Managed in a disconnected environment](#)
- [Requirements for OpenShift AI Self-Managed](#)
- [Installing the Red Hat OpenShift AI Operator](#)
- [Installing and managing Red Hat OpenShift AI components](#)
- [Adding users to OpenShift AI user groups](#)
- [Accessing the OpenShift AI dashboard](#)

3.1. REQUIREMENTS FOR OPENSIFT AI SELF-MANAGED

You must meet the following requirements before you can install Red Hat OpenShift AI on your Red Hat OpenShift cluster.

3.1.1. Platform requirements

Subscriptions * A subscription for Red Hat OpenShift AI Self-Managed is required. * If you want to install OpenShift AI Self-Managed in a Red Hat-managed cloud environment, you must also have a subscription for one of the following platforms: **Red Hat OpenShift Dedicated on Amazon Web Services (AWS)** or **Google Cloud Platform (GCP)** Red Hat OpenShift Service on Amazon Web

Services (ROSA classic) **Red Hat OpenShift Service on Amazon Web Services with hosted control planes (ROSA HCP)** Microsoft Azure Red Hat OpenShift ** Red Hat OpenShift Kubernetes Engine (OKE)

+ NOTE: While OpenShift Kubernetes Engine (OKE) typically restricts the installation of certain post-installation Operators, Red Hat provides a specific licensing exception for Red Hat OpenShift AI users. This exception exclusively applies to Operators used to support Red Hat OpenShift AI workloads. Installing or using these Operators for purposes unrelated to OpenShift AI is a violation of the OKE service agreement.

Contact your Red Hat account manager to purchase new subscriptions. If you do not yet have an account manager, complete the form at <https://www.redhat.com/en/contact> to request one.

Cluster administrator access * Cluster administrator access is required to install OpenShift AI. * You can use an existing cluster or create a new one that meets the supported version requirements.

Supported OpenShift versions The following OpenShift versions are supported for installing OpenShift AI:

- OpenShift Container Platform 4.19 to 4.20. See [OpenShift Container Platform installation overview](#).
 - To deploy models by using Distributed Inference with llm-d, your cluster must be running version 4.20 or later.
- OpenShift Dedicated 4. See [Creating an OpenShift Dedicated cluster](#).
- ROSA classic 4. See [Install ROSA classic clusters](#).
- ROSA HCP 4. See [Install ROSA with HCP clusters](#).
- OpenShift Kubernetes Engine (OKE). See [About OpenShift Kubernetes Engine](#).



NOTE

While OpenShift Kubernetes Engine (OKE) typically restricts the installation of certain post-installation Operators, Red Hat provides a specific licensing exception for Red Hat OpenShift AI users. This exception exclusively applies to Operators used to support Red Hat OpenShift AI workloads. Installing or using these Operators for purposes unrelated to OpenShift AI is a violation of the OKE service agreement.

- The following Operators are required dependencies for Red Hat OpenShift AI 2.x and 3.x. These Operators are *not supported* on OKE, but can be installed if given an exception.

Red Hat OpenShift AI version	Operator (Unsupported, Exception Required)
2.x	Authorino Operator, Service Mesh Operator, Serverless Operator

Red Hat OpenShift AI version	Operator (Unsupported, Exception Required)
3.x	Job-set-operator, openshift-custom-metrics-autoscaler-operator, cert-manager Operator, Leader Worker Set Operator, Red Hat Connectivity Link Operator, Kueue Operator (RHBOK), SR-IOV Operator, GPU Operator (with custom configurations), OpenTelemetry, Tempo, Cluster Observability Operator, IBM Spyre Operator.



NOTE

On OpenShift 4.21 and later, the OLMv1 catalog is enabled by default as a Technology Preview feature. If you plan to install dependency Operators, such as the Node Feature Discovery (NFD) Operator or the NVIDIA GPU Operator from OperatorHub, you might be redirected to a **ClusterExtensions** page instead of the standard installation form. To restore the standard installation experience, disable the OLMv1 catalog before installing these Operators. For more information, see [Troubleshooting common installation problems](#).



IMPORTANT

In OpenStack, CodeReady Containers (CRC), and other private cloud environments without integrated external DNS, you must manually configure DNS A or CNAME records after installing the Operator and components, when the LoadBalancer IP becomes available. For more information, see [Configuring External DNS for RHOAI 3.x on OpenStack and Private Clouds](#).

Cluster configuration * A minimum of 2 worker nodes with at least 8 CPUs and 32 GiB RAM each is required to install the Operator. * For single-node OpenShift clusters, the node must have at least 32 CPUs and 128 GiB RAM. * Additional resources are required depending on your workloads. * Open Data Hub must not be installed on the cluster.

Storage requirements * Your cluster must have a default storage class that supports dynamic provisioning. To confirm that a default storage class is configured, run the following command:

```
+
| oc get storageclass
```

+ If no storage class is marked as the default, see [Changing the default storage class](#) in the OpenShift Container Platform documentation.

Identity provider configuration * An identity provider must be configured for your OpenShift cluster, which provides authentication for OpenShift AI. See [Understanding identity provider configuration](#). * You must access the cluster as a user with the **cluster-admin** role; the **kubeadmin** user is not allowed. For more information, see the relevant documentation: **OpenShift Container Platform: Creating a cluster admin** OpenShift Dedicated: [Managing OpenShift Dedicated administrators](#) ** ROSA: [Creating a cluster administrator user for quick cluster access](#)

Internet access * Along with internet access, the following domains must be accessible during the installation of OpenShift AI:

- **cdn.redhat.com**
- **subscription.rhn.redhat.com**
- **registry.access.redhat.com**
- **registry.redhat.io**
- **quay.io**
 - For environments that build or customize CUDA-based images using NVIDIA's base images, or that directly pull artifacts from the NVIDIA NGC catalog, the following domains must also be accessible:
- **ngc.download.nvidia.cn**
- **developer.download.nvidia.com**



NOTE

Access to these NVIDIA domains is not required for standard OpenShift AI installations. The CUDA-based container images used by OpenShift AI are prebuilt and hosted on Red Hat's registry at **registry.redhat.io**.

Object storage * Several components of OpenShift AI require or can use S3-compatible object storage, such as AWS S3, MinIO, Ceph, or IBM Cloud Storage. Object storage provides HTTP-based access to data by using the S3 API, which is the standard interface for most object storage services.

- Object storage is required for:
 - Single-model serving platform, for storing and deploying models.
 - AI pipelines, for storing artifacts, logs, and intermediate results.
- Object storage can also be used by:
 - Workbenches, for accessing large datasets.
 - Kueue-based workloads, for reading input data and writing output results.
 - Code executed inside pipelines, for persisting generated models or other artifacts.

Custom namespaces * By default, OpenShift AI uses predefined namespaces, but you can define custom namespaces for the Operator, applications, and workbenches if needed. Namespaces created by OpenShift AI typically include **openshift** or **redhat** in their name. Do not rename these system namespaces because they are required for OpenShift AI to function properly. * If you use custom namespaces, create and label them before installing the OpenShift AI Operator. See [Configuring custom namespaces](#).

3.1.2. Component requirements

Meet the requirements for the components and capabilities that you plan to use.

Workbenches (workbenches) * To use a custom workbench namespace, create the namespace before installing the OpenShift AI Operator. See [Configuring custom namespaces](#).

AI Pipelines (aipipelines) * To store your pipeline artifacts in an S3-compatible object storage bucket so that you do not consume local storage, configure write access to your S3 bucket on your storage account. * If your cluster is running in FIPS mode, any custom container images for data science pipelines must be based on UBI 9 or RHEL 9. This ensures compatibility with FIPS-approved pipeline components and prevents errors related to mismatched OpenSSL or GNU C Library (glibc) versions. * To use your own Argo Workflows instance, after installing the OpenShift AI Operator see [Configuring pipelines with your own Argo Workflows instance](#).

Kueue-based workloads (kueue, ray, trainingoperator) * Install the Red Hat build of Kueue Operator. * Install the cert-manager Operator. * See [Configuring workload management with Kueue](#) and [Installing the distributed workloads components](#).

Trainer (trainer) * Install the JobSet Operator. For more information, see [Installing the JobSet Operator](#).

Model serving platform (kservice) * Install the cert-manager Operator. * To configure custom CPU and memory resource allocations for the OAuth proxy sidecar container in KServe inference service pods, see [Configure OAuth proxy sidecar resources for KServe](#).

Distributed Inference with llm-d (advancedkservice) * Install the cert-manager Operator. * Install the Red Hat Connectivity Link Operator. * Install the Red Hat Leader Worker Set Operator. * To use the built-in observability dashboards for Distributed Inference with llm-d deployments, install the Cluster Observability Operator. For more information, see [Installing the Cluster Observability Operator](#). * See [Deploying models by using Distributed Inference with llm-d](#).

OGX and RAG workloads (ogxoperator) * Install the OGX Operator. * Install the Red Hat OpenShift Service Mesh Operator 3.x. * Install the cert-manager Operator. * Ensure you have GPU-enabled nodes available on your cluster. * Install the Node Feature Discovery Operator. * Install the NVIDIA GPU Operator. * Configure access to S3-compatible object storage for your model artifacts. * See [Working with OGX](#).

Model registry (modelregistry) * Configure access to an external MySQL database 5.x or later; 8.x is recommended. * Configure access to S3-compatible object storage. * See [Creating a model registry](#).

3.2. CONFIGURE CUSTOM NAMESPACES

By default, OpenShift AI uses the following predefined namespaces:

- **redhat-ods-operator** contains the Red Hat OpenShift AI Operator
- **redhat-ods-applications** includes the dashboard and other required components of OpenShift AI
- **rhods-notebooks** is where basic workbenches are deployed by default

If needed, you can define custom namespaces to use instead of the predefined ones before installing OpenShift AI. This flexibility supports environments with naming policies or conventions and allows cluster administrators to control where components such as workbenches are deployed.

Namespaces created by OpenShift AI typically include **openshift** or **redhat** in their name. Do not rename these system namespaces because they are required for OpenShift AI to function properly.

Prerequisites

- You have access to an OpenShift AI cluster with cluster administrator privileges.

- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have not yet installed the Red Hat OpenShift AI Operator.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Optional: To configure a custom operator namespace:

- a. Create a namespace YAML file named **operator-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: <operator-namespace>
```

- Defines the operator namespace.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f operator-namespace.yaml
```

You see output similar to the following:

```
namespace/<operator-namespace> created
```

- c. When you install the Red Hat OpenShift AI Operator, use this namespace instead of **redhat-ods-operator**.

3. Optional: To configure a custom applications namespace:

- a. Create a namespace YAML file named **applications-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: <applications-namespace>
labels:
  opendatahub.io/application-namespace: 'true'
```

- **name** - Defines the applications namespace.
- **opendatahub.io/application-namespace** - Adds the required label.

- b. Create the namespace in your OpenShift cluster.

■

```
$ oc create -f applications-namespace.yaml
```

You see output similar to the following:

```
namespace/<applications-namespace> created
```

4. Optional: To configure a custom workbench namespace:

- a. Create a namespace YAML file named **workbench-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: <workbench-namespace>
```

- Defines the workbench namespace.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f workbench-namespace.yaml
```

You see output similar to the following:

```
namespace/<workbench-namespace> created
```

- c. When you install the Red Hat OpenShift AI components, specify this namespace for the **spec.workbenches.workbenchNamespace** field. You cannot change the default workbench namespace after you have installed the Red Hat OpenShift AI Operator.

Next step

- [Installing the Red Hat OpenShift AI Operator](#)

3.3. INSTALL THE RED HAT OPENSIFT AI OPERATOR

This section shows how to install the Red Hat OpenShift AI Operator on your OpenShift cluster using the command-line interface (CLI) and the OpenShift web console.



NOTE

If your OpenShift cluster uses a proxy to access the Internet, you can configure the proxy settings for the Red Hat OpenShift AI Operator. For more information, see *Overriding proxy settings of an Operator* in the OpenShift Container Platform documentation.

3.3.1. Installing the Red Hat OpenShift AI Operator by using the CLI

The following procedure shows how to use the OpenShift CLI (**oc**) to install the Red Hat OpenShift AI Operator on your OpenShift cluster. You must install the Operator before you can install OpenShift AI components on the cluster.

Prerequisites

- You have a running OpenShift cluster, version 4.19 or greater, configured with a default storage class that can be dynamically provisioned.
- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- If you are using custom namespaces, you have created and labeled them as required.



NOTE

The example commands in this procedure use the predefined operator namespace. If you are using a custom operator namespace, replace **redhat-ods-operator** with your namespace.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster as a cluster administrator:
 - a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - b. After you have logged in, click **Display token**.
 - c. Copy the **Log in with this token** command and paste it in your terminal.

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

3. Create a namespace for installation of the Operator by performing the following actions:



NOTE

If you have already created a custom namespace for the Operator, you can skip this step.

- a. Create a namespace YAML file named **rhods-operator-namespace.yaml**.

```
apiVersion: v1
kind: Namespace
metadata:
  name: redhat-ods-operator
```

- Defines the operator namespace.

- b. Create the namespace in your OpenShift cluster.

```
$ oc create -f rhods-operator-namespace.yaml
```

You see output similar to the following:

```
namespace/redhat-ods-operator created
```

4. Create an operator group for installation of the Operator by performing the following actions:

- a. Create an **OperatorGroup** object custom resource (CR) file, for example, **rhods-operator-group.yaml**.

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: rhods-operator
  namespace: redhat-ods-operator
```

- Defines the operator namespace.

- b. Create the **OperatorGroup** object in your OpenShift cluster.

```
$ oc create -f rhods-operator-group.yaml
```

You see output similar to the following:

```
operatorgroup.operators.coreos.com/rhods-operator created
```

5. Create a subscription for installation of the Operator by performing the following actions:

- a. Create a **Subscription** object CR file, for example, **rhods-operator-subscription.yaml**.

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: rhods-operator
  namespace: redhat-ods-operator
spec:
  name: rhods-operator
  channel: <channel>
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  startingCSV: rhods-operator.x.y.z
```

- **namespace** - Defines the operator namespace.
- **channel** - Sets the update channel. You must specify a value of **fast**, **fast-x.y**, **stable**, **stable-x.y**, **eus-x.y**, or **alpha**. For more information, see [Understanding update channels](#).
- **startingCSV** - Optional: Sets the operator version. If you do not specify a value, the subscription defaults to the latest operator version. For more information, see the [Red Hat OpenShift AI Self-Managed Life Cycle](#) Knowledgebase article.

- b. Create the **Subscription** object in your OpenShift cluster to install the Operator.

```
$ oc create -f rhods-operator-subscription.yaml
```

You see output similar to the following:

```
subscription.operators.coreos.com/rhods-operator created
```

Verification

- In the OpenShift web console, confirm that the Red Hat OpenShift AI Operator shows one of the following statuses:
 - **Installing** - installation is in progress; wait for this to change to **Succeeded**. This might take several minutes.
 - **Succeeded** - installation is successful.
The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.

Next step

- [Installing and managing Red Hat OpenShift AI components](#)

Additional resources

- [Adding users to OpenShift AI user groups](#)
- [Adding Operators to a cluster](#)

3.3.2. Installing the Red Hat OpenShift AI Operator by using the web console

The following procedure shows how to use the OpenShift web console to install the Red Hat OpenShift AI Operator on your cluster. You must install the Operator before you can install OpenShift AI components on the cluster.

Prerequisites

- You have a running OpenShift cluster, version 4.19 or greater, configured with a default storage class that can be dynamically provisioned.
- You have cluster administrator privileges for your OpenShift cluster.
- If you plan to use Llama Stack and RAG workloads, install the Red Hat OpenShift Service Mesh Operator 3.x before installing the OpenShift AI Operator. See [Installing Red Hat OpenShift Service Mesh 3.x](#).
- If you are using custom namespaces, you have created and labeled them as required.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the web console, click **Operators** → **OperatorHub**.

3. On the **OperatorHub** page, locate the Red Hat OpenShift AI Operator by scrolling through the available Operators or by typing *Red Hat OpenShift AI* into the **Filter by keyword** box.
4. Click the **Red Hat OpenShift AI** tile. The **Red Hat OpenShift AI** information pane opens.
5. Select a **Channel**. For information about subscription update channels, see [Understanding update channels](#).
6. Select a **Version**.
7. Click **Install**. The **Install Operator** page opens.
8. Review or change the selected channel and version as needed.
9. For **Installation mode**, note that the only available value is **All namespaces on the cluster (default)**. This installation mode makes the Operator available to all namespaces in the cluster.
10. For **Installed Namespace**, choose one of the following options:
 - To use the predefined operator namespace, select the **Operator recommended Namespace: redhat-ods-operator** option.
 - To use the custom operator namespace that you created, select the **Select a Namespace** option, and then select the namespace from the drop-down list.
11. For **Update approval**, select one of the following update strategies:
 - **Automatic**: New updates in the update channel are installed as soon as they become available.
 - **Manual**: A cluster administrator must approve any new updates before installation begins.



IMPORTANT

By default, the Red Hat OpenShift AI Operator follows a sequential update process. This means that if there are several versions between the current version and the target version, Operator Lifecycle Manager (OLM) upgrades the Operator to each of the intermediate versions before it upgrades it to the final, target version.

If you configure automatic upgrades, OLM automatically upgrades the Operator to the *latest* available version. If you configure manual upgrades, a cluster administrator must manually approve each sequential update between the current version and the final, target version.

For information about supported versions, see the [Red Hat OpenShift AI Life Cycle](#) Knowledgebase article.

12. Click **Install**.
The **Installing Operators** pane appears. When the installation finishes, a checkmark appears next to the Operator name.

Verification

- In the OpenShift web console, click **Operators → Installed Operators** and confirm that the Red Hat OpenShift AI Operator shows one of the following statuses:

- **Installing** - installation is in progress; wait for this to change to **Succeeded**. This might take several minutes.
- **Succeeded** - installation is successful.

Next step

- [Installing and managing Red Hat OpenShift AI components](#)

Additional resources

- [Adding users to OpenShift AI user groups](#)
- [Adding Operators to a cluster](#)

Additional resources

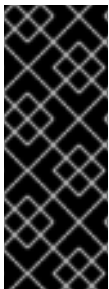
- [Overriding proxy settings of an Operator](#)

3.4. INSTALL AND MANAGE RED HAT OPENSIFT AI COMPONENTS

You can use the OpenShift command-line interface (CLI) or OpenShift web console to install and manage components of Red Hat OpenShift AI on your OpenShift cluster.

3.4.1. Installing Red Hat OpenShift AI components by using the CLI

To install Red Hat OpenShift AI components by using the OpenShift CLI (**oc**), you must create and configure a **DataScienceCluster** object.



IMPORTANT

The following procedure describes how to create and configure a **DataScienceCluster** object to install Red Hat OpenShift AI components as part of a *new* installation.

For information about changing the installation status of OpenShift AI components *after* installation, see [Updating the installation status of Red Hat OpenShift AI components by using the web console](#).

Prerequisites

- The Red Hat OpenShift AI Operator is installed on your OpenShift cluster. See [Installing the Red Hat OpenShift AI Operator](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- If you are using custom namespaces, you have created the namespaces.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster as a cluster administrator:
 - a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
 - b. After you have logged in, click **Display token**.
 - c. Copy the **Log in with this token** command and paste it in your terminal.

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

3. Create a **DataScienceCluster** object custom resource (CR) file, for example, **rhods-operator-dsc.yaml**.

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    aipipelines:
      argoWorkflowsControllers:
        managementState: Removed
      managementState: Removed
    dashboard:
      managementState: Removed
    feastoperator:
      managementState: Removed
    kserve:
      managementState: Removed
    kueue:
      managementState: Removed
      # Queue fields below take effect only when managementState is Unmanaged
      # autoCreateQueues: false
      # defaultClusterQueueName: default
      # defaultLocalQueueName: default
    ogx:
      managementState: Removed
    mlflowoperator:
      managementState: Removed
    modelregistry:
      managementState: Removed
      registriesNamespace: rhoai-model-registries
    ray:
      managementState: Removed
    trainer:
      managementState: Removed
    trainingoperator:
      managementState: Removed
    trustyai:
      managementState: Removed
```

```
workbenches:
  managementState: Removed
  workbenchNamespace: rhods-notebooks
```

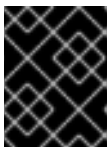
- **kueue** queue configuration fields: **autoCreateQueues**, **defaultClusterQueueName**, **defaultLocalQueueName**. These fields take effect only when **managementState** is set to **Unmanaged**. The **autoCreateQueues** field controls whether the Operator automatically creates default **ClusterQueue** and **LocalQueue** resources. Defaults to **false**. Set to **true** to enable Operator-managed queue creation.
 - **argoWorkflowsControllers.managementState**: To use your own Argo Workflows instance with the **aipipelines** component, set this to **Removed**. This allows you to integrate with a managed Argo Workflows installation already on your OpenShift cluster and avoid conflicts with the embedded controller. See *Configuring pipelines with your own Argo Workflows instance*.
 - **workbenchNamespace**: To use the predefined workbench namespace, set this value to **rhods-notebooks** or omit this line. To use a custom workbench namespace, set this value to your namespace.
4. In the **spec.components** section of the CR, for each OpenShift AI component shown, set the value of the **managementState** field to either **Managed** or **Removed**. These values are defined as follows:

Managed

The Operator actively manages the component, installs it, and tries to keep it active. The Operator will upgrade the component only if it is safe to do so.

Removed

The Operator actively manages the component but does not install it. If the component is already installed, the Operator will try to remove it.



IMPORTANT

- To learn how to install the distributed workloads components, see [Installing the distributed workloads components](#).

5. Create the **DataScienceCluster** object in your OpenShift cluster to install the specified OpenShift AI components.

```
$ oc create -f rhods-operator-dsc.yaml
```

You see output similar to the following:

```
datasciencecluster.datasciencecluster.opendatahub.io/default-dsc created
```

Verification

1. Confirm that there is at least one running pod for each component:
 - a. In the OpenShift web console, click **Workloads → Pods**.
 - b. In the **Project** list at the top of the page, select **redhat-ods-applications**.

- c. In the applications namespace, confirm that there are one or more running pods for each of the OpenShift AI components that you installed.
2. Confirm the status of all installed components:
 - a. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
 - b. Click the **Red Hat OpenShift AI Operator**.
 - c. Click the **Data Science Cluster** tab.
 - d. For the **DataScienceCluster** object called **default-dsc**, verify that the status is **Phase: Ready**.



NOTE

When you edit the **spec.components** section to change the installation status of a component, the **default-dsc** status also changes. During the initial installation, it might take a few minutes for the status phase to change from **Progressing** to **Ready**. You can access the OpenShift AI dashboard before the **default-dsc** status phase is **Ready**, but all components might not be ready.

- e. Click the **default-dsc** link to display the data science cluster details.
- f. Select the **YAML** tab.
- g. In the **status.installedComponents** section, confirm that the components you installed have a status value of **true**.



NOTE

If a component shows with the **component-name: {}** format in the **spec.components** section of the CR, the component is not installed.

3. In the OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed OpenShift AI components](#).

Next steps

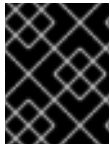
- If you are using OpenStack, CodeReady Containers (CRC), or other private cloud environments without integrated external DNS, manually configure DNS A or CNAME records after the LoadBalancer IP becomes available. For more information, see [Configuring External DNS for RHOAI 3.x on OpenStack and Private Clouds](#).
- To configure custom CPU and memory resource allocations for the OAuth proxy sidecar container in KServe inference service pods, see [Configure OAuth proxy sidecar resources for KServe](#).

- Complete any additional configuration required for the components you enabled. See the component-specific configuration sections for details.

3.4.1.1. Configure OAuth proxy sidecar resources for KServe

To prevent out-of-memory conditions when deploying large language models (LLMs), customize the CPU and memory resource requests and limits for the OAuth proxy sidecar container in KServe inference service pods.

Edit the `DataScienceCluster` custom resource (CR) to keep the KServe component in Managed state and preserve Operator reconciliation, avoiding the need to manually edit the `inferenceservice-config` ConfigMap.



IMPORTANT

This update affects running services and will rollout every inference service with authentication enabled.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- The Red Hat OpenShift AI Operator is installed.
- A **DataScienceCluster** CR exists with the KServe component in Managed state.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Edit the **DataScienceCluster** CR to add custom OAuth proxy sidecar resource values.

```
$ oc edit DataScienceCluster default-dsc
```

2. In the **spec.components.kserve** section, add the **oauthProxy.resources** field with the values that you need.

The following example increases the memory limit to **512Mi** and the memory request to **256Mi**:

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    kserve:
      managementState: "Managed"
      oauthProxy:
        resources:
          requests:
            memory: "256Mi"
            cpu: "200m"
          limits:
            memory: "512Mi"
            cpu: "500m"
```

where:

oauthProxy

Optional. Specifies the OAuth proxy sidecar configuration for KServe inference services. If you omit the **oauthProxy** block entirely, all Operator default values are preserved. If the field is absent from older **DataScienceCluster** configurations, the Operator skips the override to maintain backward compatibility.

resources.requests.memory

Specifies the minimum memory allocated to the sidecar container. The default is **64Mi**.

resources.requests.cpu

Specifies the minimum CPU allocated to the sidecar container. The default is **100m**.

resources.limits.memory

Specifies the maximum memory that the sidecar container can use. The default is **128Mi**. Increase this value if the sidecar container runs out of memory.

resources.limits.cpu

Specifies the maximum CPU that the sidecar container can use. The default is **200m**.



NOTE

Only fields that you explicitly set override the defaults. For example, specifying only **limits.memory** changes only the memory limit while preserving the default values for **requests.memory**, **requests.cpu**, and **limits.cpu**.



IMPORTANT

Resource quantities must use valid Kubernetes resource formats, for example **64Mi**, **1Gi**, **100m**, or **200m**. The API server rejects invalid values such as non-parseable strings.

3. Save and close the editor.

The Operator applies the overrides to the **inferenceservice-config** ConfigMap and automatically triggers a rollout of the **kserve-controller-manager** deployment. Existing **InferenceService** pods are rolled out with the updated resource configuration.

Verification

1. Verify that the updated values are applied to the **inferenceservice-config** ConfigMap.

```
$ oc get configmap inferenceservice-config -n redhat-ods-applications -o
jsonpath='{.data.oauthProxy}' | python3 -m json.tool
```

The output shows the updated resource values:

```
{
  "image": "<operator-managed-image>",
  "memoryRequest": "256Mi",
  "memoryLimit": "512Mi",
```

```
"cpuRequest": "100m",
"cpuLimit": "200m"
}
```

2. Confirm that the **kserve-controller-manager** deployment was rolled out.

```
$ oc rollout status deployment/kserve-controller-manager -n redhat-ods-applications
```

The output confirms a successful rollout:

```
deployment "kserve-controller-manager" successfully rolled out
```

3. Verify that new **InferenceService** pods pick up the updated sidecar resource configuration by restarting a pod or deploying a new **InferenceService** pod and inspecting the sidecar container resources.



NOTE

This step requires at least one **InferenceService** pod running in the target namespace. If no **InferenceService** is deployed, the command produces empty output.

In the following command, replace **<your_namespace>** with the namespace where your **InferenceService** is deployed, for example, a data science project namespace:

```
$ oc get pods -l serving.kserve.io/inferenceservice -n <your_namespace> -o
jsonpath='{range .items[*]}{.metadata.name}{"\n"}{range .spec.containers[*]} {.name}:
memory={.resources.limits.memory}{"\n"}{end}{end}'
```

The output shows the sidecar container resource allocations for each **InferenceService** pod:

```
my-model-predictor-00001-abcde
kserve-container: memory=4Gi
oauth-proxy: memory=512Mi
```

3.4.2. Installing Red Hat OpenShift AI components by using the web console

To install Red Hat OpenShift AI components by using the OpenShift web console, you must create and configure a **DataScienceCluster** object.



IMPORTANT

The following procedure describes how to create and configure a **DataScienceCluster** object to install Red Hat OpenShift AI components as part of a *new* installation.

- For information about changing the installation status of OpenShift AI components *after* installation, see [Updating the installation status of Red Hat OpenShift AI components by using the web console](#).

Prerequisites

- The Red Hat OpenShift AI Operator is installed on your OpenShift cluster. See [Installing the Red Hat OpenShift AI Operator](#).
- You have cluster administrator privileges for your OpenShift cluster.
- If you are using custom namespaces, you have created the namespaces.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. Click the **Red Hat OpenShift AI Operator**.
4. Click the **Data Science Cluster** tab.
5. Click **Create DataScienceCluster**.
6. For **Configure via**, select **YAML view**.

An embedded YAML editor opens showing a default custom resource (CR) for the **DataScienceCluster** object, similar to the following example:

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    aipipelines:
      argoWorkflowsControllers:
        managementState: Removed
      dashboard:
        managementState: Removed
      feastoperator:
        managementState: Removed
      kserve:
        managementState: Removed
      kueue:
        managementState: Removed
        # Queue fields below take effect only when managementState is Unmanaged
        # autoCreateQueues: false
        # defaultClusterQueueName: default
        # defaultLocalQueueName: default
      mcplifecycleoperator:
        managementState: Removed
      mlflowoperator:
        managementState: Removed
      ogx:
        managementState: Removed
      modelregistry:
        managementState: Removed
```

```
registriesNamespace: rhoai-model-registries
ray:
  managementState: Removed
trainer:
  managementState: Removed
trainingoperator:
  managementState: Removed
trustyai:
  managementState: Removed
workbenches:
  managementState: Removed
workbenchNamespace: rhods-notebooks
```

- **kueue** queue configuration fields: **autoCreateQueues**, **defaultClusterQueueName**, **defaultLocalQueueName**. These fields take effect only when **managementState** is set to **Unmanaged**. The **autoCreateQueues** field controls whether the Operator automatically creates default **ClusterQueue** and **LocalQueue** resources. Defaults to **false**. Set to **true** to enable Operator-managed queue creation.
 - **argoWorkflowsControllers.managementState**: To use your own Argo Workflows instance with the **aipipelines** component, set this to **Removed**. This allows you to integrate with a managed Argo Workflows installation already on your OpenShift cluster and avoid conflicts with the embedded controller. See *Configuring pipelines with your own Argo Workflows instance*.
 - **workbenchNamespace**: To use the predefined workbench namespace, set this value to **rhods-notebooks** or omit this line. To use a custom workbench namespace, set this value to your namespace.
7. In the **spec.components** section of the CR, for each OpenShift AI component shown, set the value of the **managementState** field to either **Managed** or **Removed**. These values are defined as follows:

Managed

The Operator actively manages the component, installs it, and tries to keep it active. The Operator will upgrade the component only if it is safe to do so.

Removed

The Operator actively manages the component but does not install it. If the component is already installed, the Operator will try to remove it.



IMPORTANT

- To learn how to install the distributed workloads components, see [Installing the distributed workloads components](#).

8. Click **Create**.

Verification

1. Confirm the status of all installed components:
 - a. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:

- On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
- b. Click the **Red Hat OpenShift AI** Operator.
 - c. Click the **Data Science Cluster** tab.
 - d. For the **DataScienceCluster** object called **default-dsc**, verify that the status is **Phase: Ready**.



NOTE

When you edit the **spec.components** section to change the installation status of a component, the **default-dsc** status also changes. During the initial installation, it might take a few minutes for the status phase to change from **Progressing** to **Ready**. You can access the OpenShift AI dashboard before the **default-dsc** status phase is **Ready**, but all components might not be ready.

- e. Click the **default-dsc** link to display the data science cluster details.
- f. Select the **YAML** tab.
- g. In the **status.installedComponents** section, confirm that the components you installed have a status value of **true**.



NOTE

If a component shows with the **component-name: {}** format in the **spec.components** section of the CR, the component is not installed.

2. Confirm that there is at least one running pod for each component:
 - a. In the OpenShift web console, click **Workloads** → **Pods**.
 - b. In the **Project** list at the top of the page, select **redhat-ods-applications** or your custom applications namespace.
 - c. In the applications namespace, confirm that there are one or more running pods for each of the OpenShift AI components that you installed.
3. In the OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed OpenShift AI components](#).

Next steps

- If you are using OpenStack, CodeReady Containers (CRC), or other private cloud environments without integrated external DNS, manually configure DNS A or CNAME records after the LoadBalancer IP becomes available. For more information, see [Configuring External DNS for RHOAI 3.x on OpenStack and Private Clouds](#).

- Complete any additional configuration required for the components you enabled. See the component-specific configuration sections for details.

3.4.3. Update the installation status of Red Hat OpenShift AI components by using the web console

You can use the OpenShift web console to update the installation status of components of Red Hat OpenShift AI on your OpenShift cluster.

Prerequisites

- The Red Hat OpenShift AI Operator is [installed](#) on your OpenShift cluster.
- You have cluster administrator privileges for your OpenShift cluster.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. Click the **Red Hat OpenShift AI Operator**.
4. Click the **Data Science Cluster** tab.
5. On the **DataScienceClusters** page, click the **default-dsc** object.
6. Click the **YAML** tab.

An embedded YAML editor opens showing the default custom resource (CR) for the **DataScienceCluster** object, similar to the following example:

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    aipipelines:
      argoWorkflowsControllers:
        managementState: Removed
      dashboard:
        managementState: Removed
      feastoperator:
        managementState: Removed
      kserve:
        managementState: Removed
      kueue:
        managementState: Removed
    # Queue fields below take effect only when managementState is Unmanaged
    # autoCreateQueues: false
    # defaultClusterQueueName: default
```

```
# defaultLocalQueueName: default
ogx:
  managementState: Removed
mlflowoperator:
  managementState: Removed
modelregistry:
  managementState: Removed
  registriesNamespace: rhoai-model-registries
ray:
  managementState: Removed
trainer:
  managementState: Removed
trainingoperator:
  managementState: Removed
trustyai:
  managementState: Removed
workbenches:
  managementState: Removed
  workbenchNamespace: rhods-notebooks
```

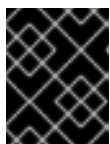
7. In the **spec.components** section of the CR, for each OpenShift AI component shown, set the value of the **managementState** field to either **Managed** or **Removed**. These values are defined as follows:

Managed

The Operator actively manages the component, installs it, and tries to keep it active. The Operator will upgrade the component only if it is safe to do so.

Removed

The Operator actively manages the component but does not install it. If the component is already installed, the Operator will try to remove it.



IMPORTANT

- To learn how to install the distributed workloads feature, see [Installing the distributed workloads components](#).

8. Click **Save**.

For any components that you updated, OpenShift AI initiates a rollout that affects all pods to use the updated image.

Verification

1. Confirm that there is at least one running pod for each component:
 - a. In the OpenShift web console, click **Workloads → Pods**.
 - b. In the **Project** list at the top of the page, select **redhat-ods-applications** or your custom applications namespace.
 - c. In the applications namespace, confirm that there are one or more running pods for each of the OpenShift AI components that you installed.
2. Confirm the status of all installed components:

3. In the OpenShift web console, go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
 - a. Click the **Red Hat OpenShift AI Operator**.
 - b. Click the **Data Science Cluster** tab and select the **DataScienceCluster** object called **default-dsc**.
 - c. Select the **YAML** tab.
 - d. In the **status.installedComponents** section, confirm that the components you installed have a status value of **true**.

**NOTE**

If a component shows with the **component-name: {}** format in the **spec.components** section of the CR, the component is not installed.

4. In the OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed OpenShift AI components](#).

3.4.4. View installed OpenShift AI components

In the Red Hat OpenShift AI dashboard, you can view a list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components.

Prerequisites

- OpenShift AI is installed in your OpenShift cluster.

Procedure

1. Log in to the OpenShift AI dashboard.
2. In the top navigation bar, click the help icon () and then select **About**.

Verification

The **About** page shows a list of the installed OpenShift AI components along with their corresponding upstream components and upstream component versions.

Additional resources

- [Installing and managing Red Hat OpenShift AI components](#)

CHAPTER 4. CONFIGURE PIPELINES WITH YOUR OWN ARGO WORKFLOWS INSTANCE

You can configure OpenShift AI to use an existing Argo Workflows instance instead of the embedded one included with AI pipelines. This configuration is useful if your OpenShift cluster already includes a managed Argo Workflows instance and you want to integrate it with OpenShift AI pipelines without conflicts. Disabling the embedded Argo Workflows controller allows cluster administrators to manage the lifecycles of OpenShift AI and Argo Workflows independently.



NOTE

You cannot enable both the embedded Argo Workflows instance and your own Argo Workflows instance on the same cluster.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed Red Hat OpenShift AI.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
4. Click the **Data Science Cluster** tab.
5. Click the default instance name (for example, **default-dsc**) to open the instance details page.
6. Click the **YAML** tab to show the instance specifications.
7. Disable the embedded Argo Workflows controllers that are managed by the OpenShift AI Operator:
 - a. In the **spec.components** section, set the value of the **managementState** field for the **aipipelines** component to **Managed**.
 - b. In the **spec.components.aipipelines** section, set the value of the **managementState** field for **argoWorkflowsControllers** to **Removed**, as shown in the following example:

Example aipipelines specification

```
# ...
spec:
  components:
    aipipelines:
      argoWorkflowsControllers:
```

```
managementState: Removed
managementState: Managed
# ...
```

8. Click **Save** to apply your changes.
9. Install and configure a compatible version of Argo Workflows on your cluster. For compatible version information, see [Supported Configurations for 3.x](#). For installation information, see the [Argo Workflows Installation documentation](#).

Verification

1. On the **Details** tab of the **DataScienceCluster** instance (for example, **default-dsc**), verify that **AIPIipelinesReady** has a **Status** of **True**.
2. Verify that the **ds-pipeline-workflow-controller** pod does not exist:
 - a. Go to **Workloads** → **Pods**.
 - b. Search for the **ds-pipeline-workflow-controller** pod.
 - c. Verify that this pod does not exist. The absence of this pod confirms that the embedded Argo Workflows controller is disabled.

CHAPTER 5. INSTALL THE DISTRIBUTED WORKLOADS COMPONENTS

To use the distributed workloads feature in OpenShift AI, you must install several components.

Prerequisites

- You have logged in to OpenShift with the **cluster-admin** role and you can access the data science cluster.
- You have installed Red Hat OpenShift AI.
- You have installed the Red Hat build of Kueue Operator on your OpenShift cluster, as described in the [Red Hat build of Kueue documentation](#).
- You have sufficient resources. In addition to the minimum OpenShift AI resources described in [Installing and deploying OpenShift AI](#) (for disconnected environments, see [Deploying OpenShift AI in a disconnected environment](#)), you need 1.6 vCPU and 2 GiB memory to deploy the distributed workloads infrastructure.
- You have installed the cert-manager Operator in OpenShift by using the web console as described in [Installing the cert-manager Operator for Red Hat OpenShift](#).
- If you want to use graphics processing units (GPUs), you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see [Enabling NVIDIA GPUs](#). If you use AMD GPUs, see [AMD GPU integration](#).



NOTE

In OpenShift AI, Red Hat supports the use of accelerators within the same cluster only.

Starting from Red Hat OpenShift AI 2.19, Red Hat supports remote direct memory access (RDMA) for NVIDIA GPUs only, enabling them to communicate directly with each other by using NVIDIA GPUDirect RDMA across either Ethernet or InfiniBand networks.

- If you want to use self-signed certificates, you have added them to a central Certificate Authority (CA) bundle as described in [Working with certificates](#) (for disconnected environments, see [Working with certificates](#)). No additional configuration is necessary to use those certificates with distributed workloads. The centrally configured self-signed certificates are automatically available in the workload pods at the following mount points:

- Cluster-wide CA bundle:

```
/etc/pki/tls/certs/odh-trusted-ca-bundle.crt
/etc/ssl/certs/odh-trusted-ca-bundle.crt
```

- Custom CA bundle:

```
/etc/pki/tls/certs/odh-ca-bundle.crt
/etc/ssl/certs/odh-ca-bundle.crt
```

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. Search for the **Red Hat OpenShift AI Operator**, and then click the Operator name to open the Operator details page.
4. Click the **Data Science Cluster** tab.
5. Click the default instance name (for example, **default-dsc**) to open the instance details page.
6. Click the **YAML** tab to show the instance specifications.
7. Enable the required distributed workloads components. In the **spec.components** section, set the **managementState** field correctly for the required components:
 - Set **kueue** to **Unmanaged** to allow the Red Hat build of Kueue Operator to manage Kueue.
 - If you want to use the Ray framework to tune models, set **ray** to **Managed**.
 - If you want to use the Kubeflow Training Operator to tune models, set **trainingoperator** to **Managed**.
 - The list of required components depends on whether the distributed workload is run from a pipeline or workbench or both, as shown in the following table.

Table 5.1. Components required for distributed workloads

Component	Pipelines only	Workbenches only	Pipelines and workbenches
dashboard	Managed	Managed	Managed
aipipelines	Managed	Removed	Managed
kueue	Unmanaged	Unmanaged	Unmanaged
ray	Managed	Managed	Managed
trainingoperator	Managed	Managed	Managed
workbenches	Removed	Managed	Managed

8. Click **Save**. After a short time, the components with a **Managed** state are ready.

Verification

Check the status of the **kubeflow-training-operator**, **kuberay-operator**, **kueue-controller-manager**, and **openshift-kueue-operator** pods, as follows:

1. In the OpenShift console, click **Workloads → Deployments**.
2. In the **Search by name** field, enter the following search strings:
 - In the **redhat-ods-applications** project, search for **kubeflow-training-operator** and **kuberay-operator**.
 - In the **openshift-kueue-operator** project, search for **kueue-controller-manager** and **openshift-kueue-operator**.
3. In each case, check the status as follows:
 - a. Click the deployment name to open the deployment details page.
 - b. Click the **Pods** tab.
 - c. Check the pod status.
When the status of the pods is **Running**, the pods are ready to use.
 - d. To see more information about each pod, click the pod name to open the pod details page, and then click the **Logs** tab.

Next step: Configure the distributed workloads feature as described in [Managing distributed workloads](#).

CHAPTER 6. ACCESS THE DASHBOARD

After you have installed OpenShift AI and added users, you can access the URL for your OpenShift AI console and share the URL with the users to let them log in and work on their models.

Prerequisites

- You have installed OpenShift AI on your OpenShift cluster.
- You have added at least one user to the user group for OpenShift AI.

Procedure

1. Log in to OpenShift web console.
2. Click the application launcher ().
3. Right-click **Red Hat OpenShift AI** and copy the URL for your OpenShift AI instance.
4. Provide this instance URL to your data scientists to let them log in to OpenShift AI.



IMPORTANT

Starting with OpenShift AI version 3.4 EA1, the dashboard URL is <https://rh-ai.apps.<cluster-domain>>. If you upgrade from an earlier version of OpenShift AI, existing bookmarks for the dashboard, including legacy **rhods-dashboard** or **data-science-gateway**, will automatically redirect to the new **rh-ai** URL format. You must provide the new URL to all Red Hat OpenShift AI administrators and users.

Verification

- Confirm that you and your users can log in to OpenShift AI by using the instance URL.

Note: In the Red Hat OpenShift AI dashboard, users can view the list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components, as described in [Viewing installed components](#).

Additional resources

- [Logging in to OpenShift AI](#)
- [Adding users to OpenShift AI user groups](#)

CHAPTER 7. ENABLE ACCELERATORS

You can enable accelerators, such as NVIDIA GPUs, Intel Gaudi AI accelerators, and AMD GPUs, to support compute-intensive workloads in OpenShift AI. Before you can use an accelerator in OpenShift AI, you must install the relevant software components. The installation process varies based on the accelerator type.

Prerequisites

- You have logged in to your OpenShift cluster.
- You have the **cluster-admin** role in your OpenShift cluster.
- You have installed an accelerator and confirmed that it is detected in your environment.

Procedure

1. Follow the appropriate documentation to enable your accelerator:
 - **NVIDIA GPUs:** See [Enabling NVIDIA GPUs](#).
 - **Intel Gaudi AI accelerators:** See [Enabling Intel Gaudi AI accelerators](#).
 - **AMD GPUs:** See [Enabling AMD GPUs](#).
 - **IBM Spyre:** See [Enabling IBM Spyre accelerators](#).
2. After installing your accelerator, create a hardware profile as described in: [Working with hardware profiles](#).

Verification

1. In the OpenShift web console, go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
2. Confirm that the following Operators appear:
 - The Operator for your accelerator
 - Node Feature Discovery (NFD)
 - Kernel Module Management (KMM)
 - The accelerator is correctly detected a few minutes after full installation of the Node Feature Discovery (NFD) and the relevant accelerator Operator. The OpenShift CLI (**oc**) displays the appropriate output for the GPU worker nodes with accelerator cards. For example, here is output confirming that an NVIDIA GPU is detected:

```
# Expected output when the accelerator is detected correctly
oc describe node <node name>
...
Capacity:
```

```
cpu:          4
ephemeral-storage: 313981932Ki
hugepages-1Gi:  0
hugepages-2Mi:  0
memory:        16076568Ki
nvidia.com/gpu: 1
pods:          250
Allocatable:
cpu:           3920m
ephemeral-storage: 288292006229
hugepages-1Gi:  0
hugepages-2Mi:  0
memory:        12828440Ki
nvidia.com/gpu: 1
pods:          250
```

CHAPTER 8. WORK WITH CERTIFICATES

When you install Red Hat OpenShift AI, OpenShift automatically applies a default Certificate Authority (CA) bundle to manage authentication for most OpenShift AI components, such as workbenches and model servers. These certificates are trusted self-signed certificates that help secure communication. However, as a cluster administrator, you might need to configure additional self-signed certificates to use some components, such as the AI pipeline server and object storage solutions. If an OpenShift AI component uses a self-signed certificate that is not part of the existing cluster-wide CA bundle, you have the following options for including the certificate:

- Add it to the OpenShift cluster-wide CA bundle.
- Add it to a custom CA bundle, separate from the cluster-wide CA bundle.

As a cluster administrator, you can also change how to manage authentication for OpenShift AI as follows:

- Manually manage certificate changes, instead of relying on the OpenShift AI Operator to handle them automatically.
- Remove the cluster-wide CA bundle, either from all namespaces or specific ones. If you prefer to implement a different authentication approach, you can override the default OpenShift AI behavior, as described in *Removing the CA bundle*.

8.1. UNDERSTANDING HOW OPENSIFT AI HANDLES CERTIFICATES

After installing OpenShift AI, the Red Hat OpenShift AI Operator automatically creates an empty **odh-trusted-ca-bundle** configuration file (ConfigMap). The Cluster Network Operator (CNO) injects the cluster-wide CA bundle into the **odh-trusted-ca-bundle** configMap with the label **"config.openshift.io/inject-trusted-cabundle"**.

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/part-of: opendatahub-operator
    config.openshift.io/inject-trusted-cabundle: 'true'
name: odh-trusted-ca-bundle
```

After the CNO operator injects the bundle, it updates the ConfigMap with the contents of the **ca-bundle.crt** file.

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/part-of: opendatahub-operator
    config.openshift.io/inject-trusted-cabundle: 'true'
name: odh-trusted-ca-bundle
data:
  ca-bundle.crt: |
    <BUNDLE OF CLUSTER-WIDE CERTIFICATES>
```

The management of CA bundles is configured through the Data Science Cluster Initialization (DSCI) object. Within this object, you can set the **spec.trustedCABundle.managementState** field to one of the following values:

- **Managed:** (Default) The Red Hat OpenShift AI Operator manages the **odh-trusted-ca-bundle** ConfigMap and adds it to all non-reserved existing and new namespaces. It does not add the ConfigMap to any reserved or system namespaces, such as **default**, **openshift-*** or **kube-***. The Red Hat OpenShift AI Operator automatically updates the ConfigMap to reflect any changes made to the **customCABundle** field.
- **Unmanaged:** The Red Hat OpenShift AI administrator manually manages the **odh-trusted-ca-bundle** ConfigMap, instead of allowing the Operator to manage it. Changing the **managementState** from **Managed** to **Unmanaged** does not remove the **odh-trusted-ca-bundle** ConfigMap. However, the ConfigMap is no longer automatically updated if changes are made to the **customCABundle** field.

The **Unmanaged** setting is useful if your organization implements a different method for managing trusted CA bundles, such as Ansible automation, and does not want the Red Hat OpenShift AI Operator to handle certificates automatically. This setting provides greater control, preventing the Operator from overwriting custom configurations.

- **Removed:** The Red Hat OpenShift AI Operator removes the **odh-trusted-ca-bundle** ConfigMap, if present, and prevents ConfigMaps from being created in new namespaces. Changing this field from **Managed** to **Removed** also deletes the ConfigMap from existing namespaces. This is the default value after upgrading Red Hat OpenShift AI from 2.7 or earlier versions to 3.5.

The **Removed** setting reduces complexity and mitigates security risks, such as unauthorized certificate changes. In high-security environments, removing the CA bundle ensures that only approved CAs are trusted, reducing the risk of cyberattacks. For example, your organization might want to restrict cluster administrators from creating trusted CA bundles to prevent OpenShift pods from communicating externally.

8.2. ADDING CERTIFICATES

If you must use a self-signed certificate that is not part of the existing cluster-wide CA bundle, you have two options for configuring the certificate:

- Add it to the cluster-wide CA bundle.
This option is useful when the certificate is needed for secure communication across multiple services or when it's required by security policies to be trusted cluster-wide. This option ensures that all services and components in the cluster trust the certificate automatically. It simplifies management because the certificate is trusted across the entire cluster, avoiding the need to configure the certificate separately for each service.
- Add it to a custom CA bundle that is separate from the OpenShift cluster-wide bundle. Consider this option for the following scenarios:
 - Limit scope: Only specific services need the certificate, not the whole cluster.
 - Isolation: Keeps custom certificates separate, preventing changes to the global configuration.
 - Avoid global impact: Does not affect services that do not need the certificate.
 - Easier management: Makes it simpler to manage certificates for specific services.

8.3. ADDING CERTIFICATES TO A CLUSTER-WIDE CA BUNDLE

You can add a self-signed certificate to a cluster-wide Certificate Authority (CA) bundle (**ca-bundle.crt**).

When the cluster-wide CA bundle is updated, the Cluster Network Operator (CNO) automatically detects the change and injects the updated bundle into the **odh-trusted-ca-bundle** ConfigMap, making the certificate available to OpenShift AI components.

Note: By default, the management state for the Trusted CA bundle is **Managed** (that is, the **spec.trustedCABundle.managementState** field in the Red Hat OpenShift AI Operator's DSCI object is set to **Managed**). If you change this setting to **Unmanaged**, you must manually update the **odh-trusted-ca-bundle** ConfigMap to include the updated cluster-wide CA bundle.

Alternatively, you can add certificates to a custom CA bundle, as described in [Adding certificates to a custom CA bundle](#).

Prerequisites

- You have created a self-signed certificate and saved the certificate to a file. For example, you have created a certificate using OpenSSL and saved it to a file named **example-ca.crt**.
- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Create a ConfigMap that includes the root CA certificate used to sign the certificate, where **</path/to/example-ca.crt>** is the path to the CA certificate bundle on your local file system:

```
oc create configmap custom-ca \
  --from-file=ca-bundle.crt=</path/to/example-ca.crt> \
  -n openshift-config
```

2. Update the cluster-wide proxy configuration with the newly-created ConfigMap:

```
oc patch proxy/cluster \
  --type=merge \
  --patch='{ "spec": { "trustedCA": { "name": "custom-ca" } } }'
```

Verification

Run the following command to verify that all non-reserved namespaces contain the **odh-trusted-ca-bundle** ConfigMap:

```
oc get configmaps --all-namespaces -l app.kubernetes.io/part-of=opendatahub-operator | grep odh-trusted-ca-bundle
```

Additional resources

- [Configuring certificates \(OpenShift Container Platform\)](#)
- [Injecting a custom CA bundle \(Red Hat OpenShift Service on AWS\)](#)
- [Injecting a custom CA bundle \(OpenShift Dedicated\)](#)

8.4. ADDING CERTIFICATES TO A CUSTOM CA BUNDLE

You can add self-signed certificates to a custom CA bundle that is separate from the OpenShift cluster-wide bundle.

This method is ideal for scenarios where components need access to external resources that require a self-signed certificate. For example, you might need to add self-signed certificates to grant AI pipelines access to S3-compatible object storage.

Prerequisites

- You have created a self-signed certificate and saved the certificate to a file. For example, you have created a certificate using OpenSSL and saved it to a file named **example-ca.crt**.
- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Log in to OpenShift.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. Click the **DSC Initialization** tab.
4. Click the **default-dsci** object.
5. Click the **YAML** tab.
6. In the **spec.trustedCABundle** section, add the custom certificate to the **customCABundle** field, as shown in the following example:

```
spec:
  trustedCABundle:
    managementState: Managed
    customCABundle: |
```

```
-----BEGIN CERTIFICATE-----
examplebundle123
-----END CERTIFICATE-----
```

7. Click **Save**.

The Red Hat OpenShift AI Operator automatically updates the ConfigMap to reflect any changes made to the **customCABundle** field. It adds the **odh-ca-bundle.crt** file containing the certificates to the **odh-trusted-ca-bundle** ConfigMap, as shown in the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app.kubernetes.io/part-of: opendatahub-operator
    config.openshift.io/inject-trusted-cabundle: 'true'
  name: odh-trusted-ca-bundle
data:
  ca-bundle.crt: |
    <BUNDLE OF CLUSTER-WIDE CERTIFICATES>
  odh-ca-bundle.crt: |
    <BUNDLE OF CUSTOM CERTIFICATES>
```

Verification

Run the following command to verify that a non-reserved namespace contains the **odh-trusted-ca-bundle** ConfigMap and that the ConfigMap contains your **customCABundle** value. In the following command, *example-namespace* is the non-reserved namespace and *examplebundle123* is the **customCABundle** value.

```
oc get configmap odh-trusted-ca-bundle -n example-namespace -o yaml | grep examplebundle123
```

8.5. USING SELF-SIGNED CERTIFICATES WITH OPENSIFT AI COMPONENTS

Some OpenShift AI components have additional options or required configuration for self-signed certificates.

8.5.1. Access S3-compatible object storage with self-signed certificates

To securely connect OpenShift AI components to object storage solutions or databases that are deployed within an OpenShift cluster that uses self-signed certificates, you must provide a certificate authority (CA) certificate. Each namespace includes a config map named **kube-root-ca.crt**, which contains the CA certificate of the internal API Server.

Use this procedure only when the object storage endpoint serves TLS by using a certificate that is signed by an internal or cluster CA. You do not need this procedure in the following cases:

- The endpoint serves plain HTTP and does not use TLS.
- The endpoint presents a certificate that the cluster-wide CA bundle already includes.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift.
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS.
- You have deployed an object storage solution or database in your OpenShift cluster.

Procedure

1. In a terminal window, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
oc login api.<cluster_name>.<cluster_domain>:6443 --web
```

2. Retrieve the current OpenShift AI trusted CA configuration and store it in a new file. This step preserves any existing custom CA bundle, so that the later steps add the internal CA for the cluster to it instead of replacing it:

```
oc get dscinitializations.dscinitialization.opendatahub.io default-dsci -o json | jq -r '.spec.trustedCABundle.customCABundle' > /tmp/my-custom-ca-bundles.crt
```

3. Add the **kube-root-ca.crt** config map for the cluster to the OpenShift AI trusted CA configuration:

```
oc get configmap kube-root-ca.crt -o jsonpath="{['data']['ca.crt']}" >> /tmp/my-custom-ca-bundles.crt
```



NOTE

The **kube-root-ca.crt** config map is identical in every namespace, so this command returns the same CA certificate regardless of your current project.

4. Update the OpenShift AI trusted CA configuration to trust certificates issued by the certificate authorities in **kube-root-ca.crt**:

```
oc patch dscinitialization default-dsci --type='json' -p='[{"op": "replace", "path": "/spec/trustedCABundle/customCABundle", "value": ""$(awk '{print "%s\\n", $0}' /tmp/my-custom-ca-bundles.crt)"}']'
```

Verification

- From a workbench pod that mounts the updated trusted CA bundle, send a test request to the HTTPS endpoint of your object storage and confirm that the TLS handshake succeeds:

```
$ oc exec -n <namespace> <workbench-pod> -- \
  curl -sS -o /dev/null -w "%{http_code}\\n" https://<s3-endpoint>
```

Any HTTP response code, such as **200** or **403**, confirms that the TLS handshake succeeded and that the cluster trusts the certificate. A **curl** error that mentions a self-signed certificate or a certificate verification failure indicates that the trusted CA bundle does not include the issuing

CA.

- A component that is configured to use the in-cluster object storage or database starts successfully. For example, a pipeline server that is configured to use an in-cluster database reaches a running state.



NOTE

For a more thorough, end-to-end check, you can confirm the certificate configuration by working through the *OpenShift AI tutorial - Fraud Detection example*, which exercises in-cluster object storage and AI pipelines against the updated trusted CA bundle.

For more information about installing local object storage buckets and creating connections, see [Running a script to install local object storage buckets and create connections](#).

For more information about enabling AI pipelines, see [Enabling pipelines](#).

8.5.2. Configure a certificate for pipelines

By default, OpenShift AI includes OpenShift cluster-wide certificates in the **odh-trusted-ca-bundle** config map. These cluster-wide certificates cover most components, such as workbenches and model servers. However, the pipeline server might require additional Certificate Authority (CA) configuration, especially when interacting with external systems that use self-signed or custom certificates.

You have the following options for adding the certificate for AI pipelines:

- Add them to the cluster-wide CA bundle, as described in [Adding certificates to a cluster-wide CA bundle](#).
- Add them to a custom bundle as described in [Adding certificates to a custom CA bundle](#).
- Provide a CA bundle that is only used for AI pipelines, as described in the following procedure.

Prerequisites

- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have created a self-signed certificate and saved the certificate to a file. For example, you have created a certificate using OpenSSL and saved it to a file named **example-ca.crt**.
- You have configured a pipeline server and it is available. For more information, see [Configuring a pipeline server](#).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Log in to the OpenShift web console as a cluster administrator.

2. Create a config map with the required bundle for your target AI pipeline:
 - a. In the **Administrator** perspective, click **Workloads → ConfigMaps**.
 - b. From the **Project** list, select the project that contains the target AI pipeline.
 - c. Click **Create ConfigMap**.
 - d. In the **Configure via** section, select the **YAML view** option.
The **Create ConfigMap** page opens, with default YAML code automatically added.
 - e. Update the default YAML code for the required bundle for your target AI pipeline:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: custom-ca-bundle
data:
  ca-bundle.crt: |
    # contents of ca-bundle.crt
```

- f. Click **Create**.
3. Update the **.spec.apiServer.cABundle** field of the underlying **DataSciencePipelinesApplication** (DSPA):
 - a. Click **Home → Search**.
 - b. From the **Resources** list, search for **DataSciencePipelinesApplication** and then select it from the list of results.
 - c. Click the **YAML** tab.
 - d. Add or update the following snippet to the **.spec.apiServer.cABundle** field:

```
apiVersion: datasciencepipelinesapplications.opendatahub.io/v1
kind: DataSciencePipelinesApplication
metadata:
  name: data-science-dspa
spec:
  apiServer:
    cABundle:
      configMapName: custom-ca-bundle
      configMapKey: ca-bundle.crt
```

where:

configMapName

Required. Specifies the name of the source config map that you created.

configMapKey

Required. Specifies the key in the source config map that holds the CA bundle.

- e. Optional: To mount the bundle at a path or file name other than the default **/dsp-custom-certs/dsp-ca.crt**, set the **caBundleFileMountPath** and **caBundleFileName** fields under **spec.apiServer**:

```
spec:
  apiServer:
    caBundleFileMountPath: /tmp/custom-dsp-certs
    caBundleFileName: custom-dsp-ca.crt
```

where:

caBundleFileMountPath

Optional. Overrides the default mount directory, **/dsp-custom-certs**.

caBundleFileName

Optional. Overrides the default file name, **dsp-ca.crt**.

- f. Click **Save** to save your changes. The pipeline server pod automatically redeploys with the updated bundle.



NOTE

The pipeline controller generates a managed config map that is named **dsp-trusted-ca-__<dspa-name>__**. For example, a DSPA named **dspa** produces a managed config map that is named **dsp-trusted-ca-dspa**. The running pipeline server pod mounts this managed config map, not any source config map that you create.

The controller creates this managed config map even when you do not set **caBundle**. When **podToPodTLS** is enabled, which is the default, the controller always adds the cluster service CA to the managed config map. When you set **caBundle**, the controller adds the CA certificates from your source config map to the same managed config map.

Verification

- In a terminal window, log in to your OpenShift cluster and confirm that the pipeline controller generated the managed CA bundle config map, which is named **dsp-trusted-ca-__<dspa-name>__**:

```
$ oc get configmap dsp-trusted-ca-<dspa-name> -n <namespace>
```

- Get the name of the pipeline API server pod, which has the **ds-pipeline-__<dspa-name>__** prefix:

```
$ oc get pods -n <namespace> | grep ds-pipeline-<dspa-name>
```

- Confirm that the CA bundle is mounted in the pipeline API server container at the default path, **/dsp-custom-certs/**:

```
$ oc exec -n <namespace> <pipeline-api-server-pod> -c ds-pipeline-api-server -- \
ls -l /dsp-custom-certs/
```

If you set **caBundleFileMountPath** or **caBundleFileName**, substitute your configured mount path and file name.

- Confirm that the bundle file contains the expected certificates:

```
$ oc exec -n <namespace> <pipeline-api-server-pod> -c ds-pipeline-api-server -- \
cat /dsp-custom-certs/dsp-ca.crt | head -5
```

The output begins with a **-----BEGIN CERTIFICATE-----** line.

- Confirm that the pipeline API server sets the related certificate environment variables:

```
$ oc exec -n <namespace> <pipeline-api-server-pod> -c ds-pipeline-api-server -- \
env | grep -iE CABUNDLE|SSL_CERT_DIR|ARTIFACT_COPY
```

The output includes **SSL_CERT_DIR**, which lists **/dsp-custom-certs** first (for example, **SSL_CERT_DIR=/dsp-custom-certs:/etc/ssl/certs:/etc/pki/tls/certs**), and the **ARTIFACT_COPY_STEP_CABUNDLE_*** variables.

8.5.3. Configure a certificate for workbenches

OpenShift AI workbenches automatically trust the cluster-wide and custom CA bundles. When you configure cluster-wide certificates, the workbench mounts the combined bundle at **/etc/pki/tls/custom-certs/ca-bundle.crt** and presets several environment variables, so that common tools and client libraries use the bundle without extra configuration. Use this procedure to apply the certificate to a workbench and, where required, to point an individual package at the bundle.

The workbench presets the following environment variables, all of which point to **/etc/pki/tls/custom-certs/ca-bundle.crt**:

SSL_CERT_FILE

The default CA bundle for OpenSSL and many Python libraries.

REQUESTS_CA_BUNDLE

The CA bundle for the Python **requests** library.

PIP_CERT

The CA bundle that **pip** uses to connect to package repositories.

GIT_SSL_CAINFO

The CA bundle that **git** uses for HTTPS connections.

KF_PIPELINES_SSL_SA_CERTS

The CA bundle that the Elyra extension reads and passes to the Kubeflow Pipelines SDK (**kfp**) when it submits pipelines to the AI pipeline server.

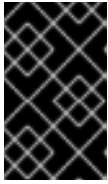
Prerequisites

- You are logged in to Red Hat OpenShift AI.
- You have configured cluster-wide certificates, as described in [Adding certificates to a cluster-wide CA bundle](#).
- You have access to a workbench in OpenShift AI.

Procedure

1. Apply the certificate to the workbench:

- For a new workbench, create the workbench after you configure cluster-wide certificates. The workbench trusts the bundle automatically. For more information about how to create workbenches, see [Creating a workbench](#).
- For an existing workbench, stop and then restart the workbench. For more information, see [Starting a workbench](#).



IMPORTANT

By default, self-signed certificates apply only to workbenches that you create after you configure cluster-wide certificates. Restarting an existing workbench applies the current bundle to it.

2. Optional: For a package that does not read any of the preset environment variables, pass the certificate path explicitly. For example, the **kfp** package connects to the AI pipeline server by using the **ssl_ca_cert** parameter:

```
from kfp.client import Client

with open(sa_token_file_path, 'r') as token_file:
    bearer_token = token_file.read()

client = Client(
    host='https://<pipeline_server_route>',
    existing_token=bearer_token,
    ssl_ca_cert='/etc/pki/tls/custom-certs/ca-bundle.crt'
)
print(client.list_experiments())
```



NOTE

The Elyra extension reads the preset **KF_PIPELINES_SSL_SA_CERTS** variable and applies this bundle automatically when you build and submit pipelines in the visual editor. When you call the **kfp** client directly, as in this example, pass **ssl_ca_cert** explicitly so that the client trusts the bundle.

Verification

- Open a terminal in the workbench and confirm that the environment variables point to the bundle file:

```
$ env | grep -E
'SSL_CERT_FILE|REQUESTS_CA_BUNDLE|PIP_CERT|GIT_SSL_CAINFO|KF_PIPELINES
_SSL_SA_CERTS'
```

Each variable points to **/etc/pki/tls/custom-certs/ca-bundle.crt**.

- Confirm that the bundle file exists and contains certificates:

```
$ head -5 /etc/pki/tls/custom-certs/ca-bundle.crt
```

The output begins with a **-----BEGIN CERTIFICATE-----** line.

8.5.4. Using the cluster-wide CA bundle for the model serving platform

By default, the model serving platform in OpenShift AI uses a self-signed certificate generated at installation for the endpoints that are created when deploying a server.

If you have configured cluster-wide certificates on your OpenShift cluster, they are used by default for other types of endpoints, such as endpoints for routes.

The following procedure explains how to use the same certificate that you already have for your OpenShift cluster.

Prerequisites

- You have cluster administrator access for the OpenShift cluster where Red Hat OpenShift AI is installed.
- You have configured cluster-wide certificates in OpenShift.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. From the list of projects, open the **openshift-ingress** project.
4. Click **YAML**.
5. Search for "cert" to find a secret with a name that includes "cert". For example, **rhods-internal-primary-cert-bundle-secret**. The contents of the secret should contain two items that are used for all OpenShift Routes: **tls.cert** (the certificate) and **tls.key** (the key).
6. Copy the reference to the secret.
7. From the list of projects, open the **istio-system** project.
8. Create a YAML file and paste the reference to the secret that you copied from the **openshift-ingress** YAML file.
9. Edit the YAML code to keep only the relevant content, as shown in the following example. Replace **rhods-internal-primary-cert-bundle-secret** with the name of your secret:

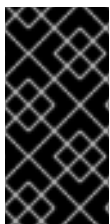
```
kind: Secret
apiVersion: v1
metadata:
  name: rhods-internal-primary-cert-bundle-secret
data:
  tls.crt: >-
    LS0tLS1CRUd...
  tls.key: >-
    LS0tLS1CRUd...
type: kubernetes.io/tls
```

10. Save the YAML file in the **istio-system** project.
11. Log in to the OpenShift web console as a cluster administrator.
12. From the OpenShift web console, go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
13. Click **Data Science Cluster**, and then click **default-dsc** → **YAML**.
14. Edit the **kserve** configuration section to refer to your secret as shown in the following example. Replace **rhods-internal-primary-cert-bundle-secret** with the name of the secret that you created in Step 8.

```
kserve:
devFlags: {}
managementState: Managed
serving:
  ingressGateway:
    certificate:
      secretName: rhods-internal-primary-cert-bundle-secret
      type: Provided
    managementState: Managed
    name: knative-serving
```

8.5.5. CA bundle configuration for OGX

By default, the OGX server image trusts only public Certificate Authorities (CAs). To enable the OGX server to communicate over TLS with external inference, embedding, or vector store providers that present self-signed certificates or certificates issued by a private CA, configure a custom CA bundle for the **OGXServer** custom resource (CR).



IMPORTANT

When you configure or change the CA bundle for a **OGXServer** CR, the OGX Operator restarts the OGX server pod so that the new certificates take effect. Plan for a brief service interruption when you apply or update the CA bundle on a **OGXServer** CR that is serving production traffic.

You have the following options for trusting a self-signed or private CA from the OGX server:

- Add the CA to the cluster-wide CA bundle.
- Add the CA to a custom CA bundle.
- Provide a CA bundle that is only used for the OGX server.

The best pattern is to add the CA to the cluster-wide or custom CA bundle and then reference the resulting **odh-trusted-ca-bundle** config map from the **OGXServer** CR. The **odh-trusted-ca-bundle** config map is automatically maintained in every non-reserved namespace and contains both the

cluster-wide CA bundle and any custom CAs that you have added through the **DSCInitialization** (DSCI) object. Alternatively, you can create a dedicated config map that contains certificates that are specific to the OGX server.

The source **ConfigMap** must use the **ogx.io/watch=true** label. The OGX Operator cache filters **ConfigMaps** with this label. Without it, the Operator cannot detect the **ConfigMap** and does not mount the CA bundle into the server pod.

When you reference a config map from the **spec.server.tlsConfig.caBundle** field of a **OGXServer** CR, the OGX Operator performs the following actions:

1. Reads the CA certificates from the source config map and validates each certificate.
2. Concatenates the valid certificates into a single bundle and stores the bundle in a managed config map that is named **<instance-name>-ca-bundle**.
3. Mounts the managed config map into the OGX server pod at **/etc/ssl/certs/ca-bundle/ca-bundle.crt**.
4. Sets the **SSL_CERT_FILE** environment variable on the server container so that TLS clients in the server use the bundle automatically.

When you change the source config map or the field reference, the OGX Operator regenerates the managed config map and restarts the OGX server pod so that the new certificates take effect.

For details about the supported **caBundle** fields, the validation rules, and the limits that the OGX Operator enforces, see *CA bundle configuration reference for OGX*.

Additional resources

- [Adding certificates to a cluster-wide CA bundle](#)
- [Adding certificates to a custom CA bundle](#)
- [Configuring a CA bundle for OGX](#)
- [CA bundle configuration reference for OGX](#)

8.5.6. Configure a CA bundle for OGX

To enable the OGX server to trust certificates that are issued by a self-signed or private Certificate Authority (CA), reference a config map that contains the CA certificates from the **spec.server.tlsConfig.caBundle** field of your **OGXServer** custom resource (CR).



IMPORTANT

When you complete this procedure, the OGX Operator restarts the OGX server pod so that the new certificates take effect. Plan for a brief service interruption when you apply or update the CA bundle on a **OGXServer** CR that is serving production traffic.

The following procedure uses the best pattern of referencing the **odh-trusted-ca-bundle** config map, which OpenShift AI automatically maintains in every non-reserved namespace. To use a dedicated config map, or to reference a config map in a different namespace, see [CA bundle configuration for OGX](#) for the alternative configuration patterns.

Prerequisites

- You have installed OpenShift 4.19 or later.
- You have logged in to Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift cluster.
- You have activated the OGX Operator in OpenShift AI. For more information, see [Activating the OGX Operator](#).
- You have deployed at least one **OGXServer** instance in OpenShift AI. For more information, see [Deploying a OGX server](#).
- You have added the required CAs to the cluster-wide CA bundle or to a custom CA bundle, as described in [Adding certificates to a cluster-wide CA bundle](#) and [Adding certificates to a custom CA bundle](#).
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform.
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS.

Procedure

1. Log in to the OpenShift AI web console as a cluster administrator.
2. From the **Project** list, select the project that contains your **OGXServer** CR.
3. Confirm that the **odh-trusted-ca-bundle** config map is present in your project:
 - a. In the **Administrator** perspective, click **Workloads → ConfigMaps**.
 - b. In the list of config maps, locate **odh-trusted-ca-bundle** and click its name.
 - c. Confirm that the config map contains the following two keys, both of which the OGX Operator reads:

ca-bundle.crt

The cluster-wide CA bundle that the Cluster Network Operator (CNO) injects.

odh-ca-bundle.crt

Custom CAs that you have added through the **customCABundle** field of the DSCI object.

4. Label the **odh-trusted-ca-bundle** in the **ConfigMap** so that the OGX Operator can detect it:

```
$ oc label configmap odh-trusted-ca-bundle ogx.io/watch=true -n <namespace>
```

The OGX Operator cache filters **ConfigMap** by the **ogx.io/watch=true** label. The Operator cannot find the **ConfigMap** without this label.

5. Reference the **odh-trusted-ca-bundle** config map from your **OGXServer** CR.
 - a. Click **Home → Search**.

- b. From the **Resources** list, search for **OGXServer** and select it. The cluster also exposes a OGX resource, which is an internal OpenShift AI resource that is managed by the Red Hat OpenShift AI Operator. Do not select **OGX**.
- c. From the list of **OGXServer** instances, click the name of the instance that you want to update.
- d. Click the **YAML** tab.
- e. Add a **tlsConfig.caBundle** field to the spec, as shown in the following example:

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: my-ogx
  namespace: my-ogx-namespace
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 1
  tlsConfig:
    caBundle:
      configMapName: odh-trusted-ca-bundle
      configMapKeys:
        - ca-bundle.crt
        - odh-ca-bundle.crt
```

- **configMapName** - Specifies the name of the source config map.
- **configMapKeys** - Specifies the keys in the source config map that the OGX Operator reads. The OGX Operator concatenates the certificates from all listed keys into a single bundle.

- f. Click **Save**. The OGX server pod automatically redeploys with the updated bundle.

Verification

1. In a terminal window, log in to your OpenShift cluster from the OpenShift CLI (**oc**):

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. Confirm that the OGX Operator created the managed CA bundle config map. The managed config map is named **<instance-name>-ca-bundle**:

```
$ oc get configmap <instance-name>-ca-bundle -n <namespace>
```

The output shows the managed config map.

3. Confirm that the **OGXServer** CR reports a successful CA bundle configuration in its status:

```
$ oc get ogxserver <instance-name> -n <namespace> -o yaml
```

In the **status.conditions** field of the output, verify that the **DeploymentReady** condition has **status: "True"** and that no condition reports a CA bundle validation failure in its **message** field.

- Confirm that the OGX server pod is running with the CA bundle mounted:

```
$ oc get pods -n <namespace> -l app.kubernetes.io/instance=<instance-name>
```

- Confirm that the CA bundle file is present in the server container at the expected mount path and that the **SSL_CERT_FILE** environment variable points to it:

```
$ oc exec -n <namespace> <pod-name> -- ls -l /etc/ssl/certs/ca-bundle/ca-bundle.crt
```

```
$ oc exec -n <namespace> <pod-name> -- printenv SSL_CERT_FILE
```

The output of the **printenv** command shows **/etc/ssl/certs/ca-bundle/ca-bundle.crt**.

- Confirm that the bundle file contains the expected CA certificates:

```
$ oc exec -n <namespace> <pod-name> -- \
  head -20 /etc/ssl/certs/ca-bundle/ca-bundle.crt
```

The output begins with a **-----BEGIN CERTIFICATE-----** line, followed by the encoded certificate data of the first certificate in the bundle.

- Confirm that the OGX server can establish a trusted TLS connection to your external endpoint by sending a test request from inside the pod:

```
$ oc exec -n <namespace> <pod-name> -- \
  curl -sS -o /dev/null -w "%{http_code}\n" <external-endpoint-url>
```

A successful HTTP status code, such as **200**, indicates that the certificate chain is validated. A **curl** error that mentions a self-signed certificate or a certificate verification failure indicates that the bundle does not include the correct issuing CA.

8.5.7. CA bundle configuration for OGX

Use this reference to look up the supported **subfields** of **spec.server.tlsConfig.caBundle** on a **OGXServer** custom resource (CR), the alternative configuration patterns, the validation rules and limits that the OGX Operator enforces, the conditions that cause CA bundle validation to fail, and the bundle path that client code can use to establish trusted TLS connections from inside the OGX server pod.

8.5.7.1. caBundle subfields

The **spec.server.tlsConfig.caBundle** field accepts the following **subfields**:

configMapName

Required. The name of the source config map that contains the CA certificates.

configMapNamespace

Optional. The namespace of the source config map. If you omit this field, the OGX Operator reads the config map from the namespace of the **OGXServer** CR. Cross-namespace references require that the OGX Operator service account has read access to the source config map.

configMapKeys

Optional. A list of keys in the source config map that contain CA bundles. The OGX Operator reads every listed key and concatenates the certificates into a single bundle. If you omit this field, the OGX Operator reads only the default key, **ca-bundle.crt**. Set **configMapKeys** when the source config map holds CA data under one or more keys with names other than the default, for example, **odh-trusted-ca-bundle**, which holds CA data under both **ca-bundle.crt** and **odh-ca-bundle.crt**.

8.5.7.2. Configuration examples

The following examples show the alternative patterns for referencing a source config map. For the best pattern, which uses the **odh-trusted-ca-bundle** config map, see [Configuring a CA bundle for OGX](#).

Referencing a dedicated config map

To use a config map that contains certificates that are specific to the OGX server, create the config map in the same namespace as the **OGXServer** CR and reference it by name. The following example references a dedicated config map that contains a single CA bundle in the default **ca-bundle.crt** key:

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: my-ogx
  namespace: my-ogx-namespace
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 1
  tlsConfig:
    caBundle:
      configMapName: external-llm-ca
```

+ * **configMapName** - Specifies the name of the dedicated config map that you created. Because no **configMapKeys** value is set, the OGX Operator reads the default key, **ca-bundle.crt**.

To create a dedicated config map, run the following command, in which **<ca-bundle-configmap>** is the name of the config map to create, **<path/to/ca-bundle.crt>** is the path to a file on your local file system that contains one or more PEM-encoded CA certificates, and **<namespace>** is the namespace that contains your **OGXServer** CR:

```
$ oc create configmap <ca-bundle-configmap> \
  --from-file=ca-bundle.crt=<path/to/ca-bundle.crt> \
  -n <namespace>
```

To include multiple CA certificates in the dedicated config map, concatenate their PEM blocks in the file before you create the config map.

Referencing a config map in a different namespace

To reference a config map in a namespace other than the namespace of the **OGXServer** CR, set the **configMapNamespace** field. Cross-namespace references require that the OGX Operator service account has read access to the source config map. The following example references a config map named **enterprise-ca-bundle** in the **security-system** namespace:

```
spec:
```

```

server:
  tlsConfig:
    caBundle:
      configMapName: enterprise-ca-bundle
      configMapNamespace: security-system

```

+ * **configMapName** - Specifies the name of the source config map. * **configMapNamespace** - Specifies the namespace that contains the source config map.

8.5.7.3. Validation rules

The OGX Operator processes only PEM blocks of type **CERTIFICATE**. PEM blocks of other types, such as **PRIVATE KEY**, are ignored without error. Each block must parse as a valid X.509 certificate. The combined bundle must contain at least one valid **CERTIFICATE** block; if no valid certificates are found, the OGX Operator does not create the managed config map and reports a validation failure on the CR status.

8.5.7.4. Limits

The OGX Operator enforces the following limits:

Table 8.1. CA bundle limits

Limit	Value	Description
Maximum bundle size	10 MB	The total size of the concatenated PEM bundle that the OGX Operator generates from the selected keys. Bundles that exceed this size are rejected.
Maximum certificate count	1000	The maximum number of valid X.509 CERTIFICATE blocks that the OGX Operator processes. Bundles that contain more certificates are rejected.
Accepted PEM block type	CERTIFICATE	Only PEM blocks of type CERTIFICATE are processed. Blocks of any other type are ignored without error.

The standard 1 MB Kubernetes config map size limit applies to the source config map. The 10 MB limit applies to the concatenated bundle that the OGX Operator builds from the selected keys.

8.5.7.5. CA bundle validation failure conditions

When a CA bundle validation error occurs, the OGX Operator does not deploy the OGX server pod with an invalid bundle. Instead, the OGX Operator surfaces the error on the **status.conditions** field of the **OGXServer** CR.

A **OGXServer** CR publishes the following condition types:

DeploymentReady

Set to **"True"** when the OGX server deployment has been created and rolled out successfully. CA bundle validation errors cause the OGX Operator to set this condition to **"False"**, with a **message** field that describes the underlying error.

ServiceReady

Set to **"True"** when the OGX server service is available.

HealthCheck

Set to **"True"** when the OGX server passes its readiness checks.

The CR also publishes a **status.phase** field that summarizes the overall lifecycle state of the **OGXServer** instance, with values such as **Initializing**, **Ready**, and **Failed**.

The following conditions cause a CA bundle validation failure:

- The referenced config map does not exist in the specified namespace.
- The selected key does not exist in the source config map.
- The selected keys do not contain any valid X.509 **CERTIFICATE** PEM blocks.
- The concatenated bundle exceeds the 10 MB size limit.
- The concatenated bundle contains more than 1000 certificates.
- The OGX Operator service account does not have read access to the source config map.

After you correct the source config map or the CR reference, the OGX Operator reconciles the change automatically and updates the managed config map.

8.5.7.6. Using the CA bundle from client code

After the OGX Operator mounts the CA bundle into the OGX server pod, the bundle is available to client code that runs inside the pod at the following path:

```
/etc/ssl/certs/ca-bundle/ca-bundle.crt
```

The OGX Operator also sets the **SSL_CERT_FILE** environment variable on the server container to point to this path. Most Python HTTP libraries, including **httpx** and **requests**, honor **SSL_CERT_FILE** automatically and use the bundle without further configuration.

If your client code uses a library that does not honor **SSL_CERT_FILE**, pass the bundle path explicitly. For example, the OGX Python client accepts a custom certificate bundle through its TLS configuration:

```
from ogx_client import OgxClient
import httpx

http_client = httpx.Client(verify="/etc/ssl/certs/ca-bundle/ca-bundle.crt")
client = OgxClient(
    base_url="https://my-ogx.my-ogx-namespace.svc:8321",
    http_client=http_client,
)
```

8.6. MANAGE CERTIFICATES WITHOUT THE RED HAT OPENSIFT AI OPERATOR

By default, the Red Hat OpenShift AI Operator manages the **odh-trusted-ca-bundle** config map, which contains the trusted CA bundle and is applied to all non-reserved namespaces in the cluster. The Operator automatically updates this config map whenever changes are made to the CA bundle.

If your organization prefers to manage trusted CA bundles independently, for example, by using Ansible automation, you can disable this default behavior to prevent automatic updates by the Red Hat OpenShift AI Operator.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift.
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS.

Procedure

1. In the OpenShift web console, go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
2. Click the **Red Hat OpenShift AI Operator**.
3. Click the **DSC Initialization** tab.
4. Click the **default-dsci** object.
5. Click the **YAML** tab.
6. In the **spec** section, change the value of the **managementState** field for **trustedCABundle** to **Unmanaged**, as shown:

```
spec:
  trustedCABundle:
    managementState: Unmanaged
```

7. Click **Save**.
Changing the **managementState** from **Managed** to **Unmanaged** prevents automatic updates when the **customCABundle** field is modified, but does not remove the **odh-trusted-ca-bundle** config map.

Verification

- In the **spec** section, set the **customCABundle** field to a test value (for example, **example123**), and then click **Save**:

```
spec:
  trustedCABundle:
    managementState: Unmanaged
    customCABundle: example123
```

- Confirm that your test value is not propagated while the trusted CA bundle is **Unmanaged**. Display the **odh-ca-bundle.crt** key of the **odh-trusted-ca-bundle** config map in any non-reserved namespace, and confirm that it does not contain the test value:

```
$ oc get configmap odh-trusted-ca-bundle -n <namespace> -o jsonpath={.data.odh-ca-bundle\.crt}
```

Because the trusted CA bundle is **Unmanaged**, the Operator does not reconcile the bundle, so the output does not include the test value.

8.7. REMOVING THE CA BUNDLE

If you prefer to implement a different authentication approach for your OpenShift AI installation, you can override the default behavior by removing the CA bundle.

You have two options for removing the CA bundle:

- Remove the CA bundle from all non-reserved projects in OpenShift AI.
- Remove the CA bundle from a specific project.

8.7.1. Remove the CA bundle from all namespaces

You can remove a Certificate Authority (CA) bundle from all non-reserved namespaces in OpenShift AI. This process changes the default configuration and disables the creation of the **odh-trusted-ca-bundle** config map, as described in [Working with certificates](#) (OpenShift AI Self-Managed) or [Working with certificates](#) (OpenShift AI Self-Managed in a disconnected environment).



NOTE

The **odh-trusted-ca-bundle** config maps are only deleted from namespaces when you set the **managementState** of **trustedCABundle** to **Removed**; deleting the DSC Initialization does not delete the config maps.

To remove a CA bundle from a single namespace only, see [Removing the CA bundle from a single namespace](#) (OpenShift AI Self-Managed) or [Removing the CA bundle from a single namespace](#) (OpenShift AI Self-Managed in a disconnected environment).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform.
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem → Installed Operators**.
 - On OpenShift 4.19, click **Operators → Installed Operators**.
3. Click the **Red Hat OpenShift AI** Operator.
4. Click the **DSC Initialization** tab.
5. Click the **default-dsci** object.
6. Click the **YAML** tab.
7. In the **spec** section, change the value of the **managementState** field for **trustedCABundle** to **Removed**:

```
spec:
  trustedCABundle:
    managementState: Removed
```

8. Click **Save**.

Verification

- Run the following command to verify that the **odh-trusted-ca-bundle** config map has been removed from all namespaces:

```
$ oc get configmaps --all-namespaces | grep odh-trusted-ca-bundle
```

The command should not return any config maps.

- Confirm that the Operator does not create the **odh-trusted-ca-bundle** config map in new namespaces while **managementState** is **Removed**. Create a test namespace, and then check for the config map:

```
$ oc create namespace <test-namespace>

$ oc get configmap odh-trusted-ca-bundle -n <test-namespace>
```

The **oc get** command reports that the config map is not found, which confirms that the Operator no longer injects the bundle.

- Delete the test namespace when you finish:

```
$ oc delete namespace <test-namespace>
```

8.7.2. Remove the CA bundle from a single namespace

You can remove a custom Certificate Authority (CA) bundle from individual namespaces in OpenShift AI. This process disables the creation of the **odh-trusted-ca-bundle** config map for the specified namespace only.

To remove a CA bundle from all namespaces, see [Removing the CA bundle from all namespaces](#) (OpenShift AI Self-Managed) or [Removing the CA bundle from all namespaces](#) (OpenShift AI Self-Managed in a disconnected environment).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

- Run the following command to remove a CA bundle from a namespace. In the following command, *example-namespace* is the non-reserved namespace.

```
$ oc annotate ns example-namespace security.opendatahub.io/inject-trusted-ca-bundle=false
```

Verification

- Run the following command to verify that the CA bundle has been removed from the namespace. In the following command, *example-namespace* is the non-reserved namespace.

```
$ oc get configmap odh-trusted-ca-bundle -n example-namespace
```

The command should return **configmaps "odh-trusted-ca-bundle" not found**.

- To restore the **odh-trusted-ca-bundle** config map in the namespace, remove the annotation. The Operator recreates the config map. In the following command, *example-namespace* is the non-reserved namespace.

```
$ oc annotate ns example-namespace security.opendatahub.io/inject-trusted-ca-bundle-  
$ oc get configmap odh-trusted-ca-bundle -n example-namespace
```

The **oc get** command shows the recreated config map.

CHAPTER 9. VIEW LOGS AND AUDIT RECORDS

As a cluster administrator, you can use the OpenShift AI Operator logger to monitor and troubleshoot issues. You can also use OpenShift audit records to review a history of changes made to the OpenShift AI Operator configuration.

9.1. CONFIGURE THE OPENSIFT AI OPERATOR LOGGER

You can change the log level for the OpenShift AI Operator by setting the `.spec.devFlags.logLevel` flag for the **DSC Initialization (DSCI)** custom resource during runtime. If you do not set a `logLevel` value, the logger uses the **info** log level by default.

The log level that you set with `.spec.devFlags.logLevel` applies only to the **rhods-operator** itself, not to the individual OpenShift AI components that the Operator manages.

The following table describes the available log levels:

Table 9.1. Available log levels for the OpenShift AI Operator

Log level	Severity	Verbosity	Output format	Description
error	N/A	Low	JSON	Restricts logging output to error messages only.
info	0	Medium	JSON	Enables standard informational logs and error . Default when not set.
debug	-1	High	JSON	Enables all logs: debug , info , and error .
Custom numeric	1 or higher	Very High	JSON	Fine-grained controller execution logging for advanced troubleshooting. Higher integers yield progressively more detailed tracing output.

Prerequisites

- You have administrator access to the **DSCInitialization** resources in the OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform

- [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Log in to the OpenShift as a cluster administrator.
2. Go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
3. Click the **DSC Initialization** tab.
4. Click the **default-dsci** object.
5. Click the **YAML** tab.
6. In the **spec** section, update the **.spec.devFlags.logLevel** flag with the log level that you want to set.

```
apiVersion: dscinitialization.opendatahub.io/v2
kind: DSCInitialization
metadata:
  name: default-dsci
spec:
  devFlags:
    logLevel: debug
```

7. Click **Save**.
8. **Alternatively**, to configure the log level from the OpenShift CLI (**oc**), run the following command:

```
$ oc patch dsci default-dsci -p {"spec":{"devFlags":{"logLevel":"debug"}}} --
type=merge
```

Verification

- If you set the log level to **error**, logs generate infrequently and only capture critical execution failures and errors.
- If you set the log level to **info** (or leave it unset), logs generate at a standard frequency and include **info** and **error** messages.
- If you set the log level to **debug**, logs generate often and include all **debug**, **info**, and **error** messages.
- If you set the log level to a numeric value (such as **1**, **2**, or **3**), logs generate at an extremely high frequency, exposing deep, fine-grained controller execution data for advanced troubleshooting. Higher integers yield progressively more detailed tracing output.

View the OpenShift AI Operator logs

1. Log in to the OpenShift CLI (**oc**).

2. Run the following command to stream logs from all Operator pods:

```
for pod in $(oc get pods -l name=rhods-operator -n redhat-ods-operator -o name); do  
oc logs -f "$pod" -n redhat-ods-operator & done
```

The Operator pod logs open in your terminal.

TIP

Press **Ctrl+C** to stop viewing. To fully stop all log streams, run **kill \$(jobs -p)**.

You can also view each Operator pod log in the OpenShift console by navigating to **Workloads → Pods**, selecting the **redhat-ods-operator** project, clicking a pod name, and then clicking the **Logs** tab.

9.2. VIEW AUDIT RECORDS

Cluster administrators can use OpenShift auditing to see changes made to the OpenShift AI Operator configuration by reviewing modifications to the DataScienceCluster (DSC) and DSCInitialization (DSCI) custom resources. Audit logging is enabled by default in standard OpenShift cluster configurations. For more information, see [Viewing audit logs](#) in the OpenShift documentation.



NOTE

In Red Hat OpenShift Service on AWS, audit logging is disabled by default because the Elasticsearch log store does not provide secure storage for audit logs. To configure log forwarding, see [Logging](#) in the Red Hat OpenShift Service on AWS documentation.

The following example shows how to use the OpenShift audit logs to see the history of changes made (by users) to the DSC and DSCI custom resources.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. To access the full content of the changed custom resources, set the OpenShift audit log policy to **WriteRequestBodies** or a more comprehensive profile. For more information, see [Configuring the audit log policy](#).
3. Fetch the audit log files that are available for the relevant control plane nodes. For example:

```
oc adm node-logs --role=master --path=kube-apiserver/ \  
| awk '{ print $1 }' | sort -u \  
| while read node ; do  
    oc adm node-logs $node --path=kube-apiserver/audit.log < /dev/null  
done \  
| grep opendatahub > /tmp/kube-apiserver-audit-opendatahub.log
```

4. Search the files for the DSC and DSCI custom resources. For example:

```
jq 'select((.objectRef.apiGroup == "dscinitialization.opendatahub.io"  
    or .objectRef.apiGroup == "datasciencecluster.opendatahub.io")  
    and .user.username != "system:serviceaccount:redhat-ods-operator:redhat-ods-  
operator-controller-manager"  
    and .verb != "get" and .verb != "watch" and .verb != "list")' < /tmp/kube-apiserver-  
audit-opendatahub.log
```

Verification

- The commands return relevant log entries.

TIP

To configure the log retention time, see the [Logging](#) section in the OpenShift documentation.

CHAPTER 10. TROUBLESHOOTING REFERENCE: INSTALLATION

If you are experiencing difficulties installing the Red Hat OpenShift AI Operator, read this section to understand what could be causing the problem and how to resolve it.

If the problem is not included here or in the release notes, contact [Red Hat Support](#). When opening a support case, it is helpful to include debugging information about your cluster.

You can collect this information by using the **must-gather** tool as described in [Must-Gather for Red Hat OpenShift AI](#) and [Gathering data about your cluster](#).

You can also adjust the log level of OpenShift AI Operator components to increase or reduce log verbosity to suit your use case. For more information, see [Configuring the OpenShift AI Operator logger](#).

10.1. INSTALLING CERTAIN OPERATORS REDIRECTS TO CLUSTEREXTENSIONS PAGE

Problem

On OpenShift 4.21 and later, clicking **Install** on Operators such as the Node Feature Discovery (NFD) Operator or the NVIDIA GPU Operator in OperatorHub redirects to a **ClusterExtensions** page instead of the standard installation flow. This occurs because OLMv1 catalog is enabled by default as a Technology Preview feature on OpenShift 4.21+.

When OLMv1 is enabled, the following behavior occurs:

- You must manually create **ClusterExtension** custom resources using YAML to install Operators through OLMv1.
- OpenShift AI and dependency Operators installed using CSV-based OLMv0 do not appear under **Ecosystem → Installed Operators**, although **oc get csv** lists them correctly.

Diagnosis

1. In the OpenShift web console, navigate to **Operators → OperatorHub**.
2. Search for "Node Feature Discovery" or "NVIDIA GPU Operator".
3. Click the Operator tile and then click **Install**.
4. If you are redirected to a **ClusterExtensions** page instead of the standard installation form, OLMv1 catalog is enabled.

Resolution

Disable the OLMv1 catalog to restore the standard installation experience for dependency Operators. Choose one of the following methods:

Method 1: User Preferences

1. In the OpenShift web console, click your username in the upper-right corner.
2. Select **User Preferences**.

3. Under **OLMv1 Catalog**, clear the **Enable OLMv1 catalog** checkbox.
4. Return to **Operators → OperatorHub** and install the Operator, such as the Node Feature Discovery (NFD) or NVIDIA GPU, using the standard installation flow.

Method 2: Software Catalog filter

1. In the OpenShift web console, navigate to **Operators → OperatorHub**.
2. Under **Types** on the left sidebar, click **Operators**.
3. Clear the **Enable OLMv1 Tech Preview** checkbox.
4. Install the Operator, such as the Node Feature Discovery (NFD) or NVIDIA GPU, using the standard installation flow.

After disabling OLMv1, the Operators install using CSV-based OLMv0, and appear correctly in **Operators → Installed Operators**.



NOTE

To view Operators installed using OLMv0 while OLMv1 is enabled, access the legacy view directly at `/k8s/ns/<_namespace_>/clusterserviceversions`, replacing `<namespace>` with the Operator namespace such as `openshift-nfd` or `nvidia-gpu-operator`.

10.2. THE RED HAT OPENSIFT AI OPERATOR CANNOT BE RETRIEVED FROM THE IMAGE REGISTRY

Problem

When attempting to retrieve the Red Hat OpenShift AI Operator from the image registry, an **Failure to pull from quay** error message appears. The Red Hat OpenShift AI Operator might be unavailable for retrieval in the following circumstances:

- The image registry is unavailable.
- There is a problem with your network connection.
- Your cluster is not operational and is therefore unable to retrieve the image registry.

Diagnosis

Check the logs in the **Events** section in OpenShift for further information about the **Failure to pull from quay** error message.

Resolution

- Contact Red Hat support.

10.3. OPENSIFT AI DOES NOT INSTALL ON UNSUPPORTED INFRASTRUCTURE

Problem

You are deploying on an environment that is not documented as supported by the Red Hat OpenShift AI Operator.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Deploying on \$infrastructure, which is not supported. Failing Installation** error message.

Resolution

- Before proceeding with a new installation, ensure that you have a fully supported environment on which to install OpenShift AI. For more information, see [Supported Configurations for 3.x](#).

10.4. THE CREATION OF THE OPENSIFT AI CUSTOM RESOURCE (CR) FAILS

Problem

During the installation process, the OpenShift AI Custom Resource (CR) does not get created. This issue occurs in unknown circumstances.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column.
The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Attempt to create the ODH CR failed.** error message.

Resolution

- Contact Red Hat support.

10.5. THE CREATION OF THE OPENSIFT AI NOTEBOOKS CUSTOM RESOURCE (CR) FAILS

Problem

During the installation process, the OpenShift AI Notebooks Custom Resource (CR) does not get created. This issue occurs in unknown circumstances.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-*<random string>*** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Attempt to create the RHODS Notebooks CR failed.** error message.

Resolution

- Contact Red Hat support.

10.6. THE OPENSIFT AI DASHBOARD IS NOT ACCESSIBLE

Problem

After installing OpenShift AI, the **redhat-ods-applications**, **redhat-ods-monitoring**, and **redhat-ods-operator** project namespaces are **Active** but you cannot access the dashboard due to an error in one of the pods.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads → Pods**.
3. Set the **Project** to **All Projects**.
4. Click **Filter** and select the checkbox for every status except **Running** and **Completed**. The page displays the pods that have an error.

Resolution

- To see more information and troubleshooting steps for a pod, on the **Pods** page, click the link in the **Status** column for the pod.
- If the **Status** column does not display a link, click the pod name to open the pod details page and then click the **Logs** tab.

10.7. REINSTALLING OPENSIFT AI FAILS WITH AN ERROR

Problem

After uninstalling the OpenShift AI Operator and reinstalling it by using the CLI, the reinstallation fails with an **unable to find DSCInitialization** error in one of the OpenShift AI Operator pod logs. This issue can occur if the **Auth** custom resource from the previous installation was not deleted after uninstalling the OpenShift AI Operator and before reinstalling it. For more information, see [Understanding the uninstallation process](#).

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads** → **Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-`<random string>`** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for an error message similar to the following:

```
{
  "name": "auth",
  "namespace": "",
  "name": "auth",
  "reconcileID": "7bff53ae-1252-46fe-831a-fdc824078a1b",
  "error": "unable to find DSCInitialization",
  "stacktrace": "sigs.k8s.io/controller-runtime/pkg/internal/controller."
}
```

Resolution

1. Uninstall the OpenShift AI Operator.
2. Delete the **Auth** custom resource:
 - a. In the OpenShift web console, switch to the **Administrator** perspective.
 - b. Click **API Explorer**.
 - c. From the **All groups** drop-down list, select or enter **services.platform.opendatahub.io**.
 - d. Click the **Auth** kind.
 - e. Click the **Instances** tab.
 - f. Click the action menu (⋮) and select **Delete Auth**. The **Delete Auth** dialog appears.
 - g. Click **Delete**.
3. Install the OpenShift AI Operator again.

10.8. THE DEDICATED-ADMINS ROLE-BASED ACCESS CONTROL (RBAC) POLICY CANNOT BE CREATED

Problem

The Role-based access control (RBAC) policy for the dedicated-admins group in the target project cannot be created. This issue occurs in unknown circumstances.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads** → **Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-<random string>** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Attempt to create the RBAC policy for dedicated admins group in \$target_project failed.** error message.

Resolution

- Contact Red Hat support.

10.9. THE ODH PARAMETER SECRET DOES NOT GET CREATED

Problem

An issue with the OpenShift AI Operator's flow could result in failure to create the ODH parameter.

Diagnosis

1. In the OpenShift web console, switch to the **Administrator** perspective.
2. Click **Workloads** → **Pods**.
3. Set the **Project** to **All Projects** or **redhat-ods-operator**.
4. Click the **rhods-operator-<random string>** pod that shows an error in the **Status** column. The **Pod details** page appears.
5. Click **Logs**.
6. Select **rhods-operator** from the drop-down list.
7. Check the log for the **ERROR: Addon managed odh parameter secret does not exist.** error message.

Resolution

- Contact Red Hat support.

CHAPTER 11. UNINSTALL RED HAT OPENSIFT AI SELF-MANAGED

This section shows how to use the OpenShift CLI (**oc**) to uninstall the Red Hat OpenShift AI Operator and any OpenShift AI components installed and managed by the Operator.



NOTE

Using the OpenShift CLI (**oc**) is the recommended way to uninstall the Operator. Depending on your version of OpenShift, using the web console to perform the uninstallation might not prompt you to uninstall all associated components. This could leave you unclear about the final state of your cluster.

11.1. UNDERSTANDING THE UNINSTALLATION PROCESS

Installing Red Hat OpenShift AI created several custom resource instances on your OpenShift cluster for various components of OpenShift AI. After installation, users likely created several additional resources while using OpenShift AI. Uninstalling OpenShift AI removes the resources that were created by the Operator, but retains the resources created by users to prevent inadvertently deleting information you might want.

What is deleted

Uninstalling OpenShift AI removes the following resources from your OpenShift cluster:

- **DataScienceCluster** custom resource instance and the custom resource instances it created for each component
- **DSCInitialization** custom resource instance
- **Auth** custom resource instance created during or after installation
- **FeatureTracker** custom resource instances created during or after installation
- **ServiceMesh** custom resource instance created by the Operator during or after installation
- **KNativeServing** custom resource instance created by the Operator during or after installation
- **redhat-ods-applications**, **redhat-ods-monitoring**, and **rhods-notebooks** namespaces created by the Operator
- Workloads in the **rhods-notebooks** namespace
- **Subscription**, **ClusterServiceVersion**, and **InstallPlan** objects
- **KfDef** object (version 1 Operator only)

What might remain

Uninstalling OpenShift AI retains the following resources in your OpenShift cluster:

- Projects created by users
- Custom resource instances created by users

- Custom resource definitions (CRDs) created by users or by the Operator

While these resources might still remain in your OpenShift cluster, they are not functional. After uninstalling, Red Hat recommends that you review the projects and custom resources in your OpenShift cluster and delete anything no longer in use to prevent potential issues, such as pipelines that cannot run, notebooks that cannot be undeployed, or models that cannot be undeployed.

Additional resources

- [Operator Lifecycle Manager \(OLM\) uninstall documentation](#)

11.2. UNINSTALLING OPENSIFT AI SELF-MANAGED BY USING THE CLI

The following procedure shows how to use the OpenShift CLI (**oc**) to uninstall the Red Hat OpenShift AI Operator and any OpenShift AI components installed and managed by the Operator.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have backed up the persistent disks or volumes used by your persistent volume claims (PVCs).

Procedure

1. Open a new terminal window.
2. Log in to your OpenShift cluster as a cluster administrator, as shown in the following example:

```
$ oc login <openshift_cluster_url> -u system:admin
```

3. Create a **ConfigMap** object for deletion of the Red Hat OpenShift AI Operator.

```
$ oc create configmap delete-self-managed-odh -n redhat-ods-operator
```

4. To delete the **rhods-operator**, set the **addon-managed-odh-delete** label to **true**.

```
$ oc label configmap/delete-self-managed-odh api.openshift.com/addon-managed-odh-delete=true -n redhat-ods-operator
```

5. When all objects associated with the Operator are removed, delete the **redhat-ods-operator** project.
 - a. Set an environment variable for the **redhat-ods-applications** project.

```
$ PROJECT_NAME=redhat-ods-applications
```

-
- b. Wait until the **redhat-ods-applications** project has been deleted.

```
while oc get project $PROJECT_NAME &> /dev/null; do
  echo "The $PROJECT_NAME project still exists"
  sleep 1
done
echo "The $PROJECT_NAME project no longer exists"
```

When the **redhat-ods-applications** project has been deleted, you see the following output.

```
The redhat-ods-applications project no longer exists
```

- c. When the **redhat-ods-applications** project has been deleted, delete the **redhat-ods-operator** project.

```
$ oc delete namespace redhat-ods-operator
```

Verification

1. Confirm that the **rhods-operator** subscription no longer exists.

```
$ oc get subscriptions --all-namespaces | grep rhods-operator
```

2. Confirm that the following projects no longer exist.

- **redhat-ods-applications**
- **redhat-ods-monitoring**
- **redhat-ods-operator**
- **rhods-notebooks**

```
$ oc get namespaces | grep -e redhat-ods* -e rhods*
```

The **rhods-notebooks** project existed only if you installed the workbenches component of OpenShift AI. See [Installing and managing Red Hat OpenShift AI components](#).